



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

TOOL COLABORATIV SEMI-AUTOMAT DE ADNOTARE

LUCRARE DE LICENȚĂ

Absolvent: **Victor-Andrei GRIGORAȘ**

Coordonator **SL. Dr. Ing. Mircea-Paul MUREȘAN**
științific:

2026



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN,
Prof. dr. ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Victor-Andrei GRIGORAȘ**

TOOL COLABORATIV SEMI-AUTOMAT DE ADNOTARE

1. **Enunțul temei:** *Dezvoltarea unui instrument de adnotare vizuală semi-automat, asistat de Inteligență Artificială, cu capabilități de lucru colaborativ, pentru eficientizarea procesului de etichetare și reducerea timpului și a efortului.*
2. **Conținutul lucrării:** *Pagina de prezentare, introducere, obiectivele lucrării, studiu bibliografic, analiză și fundamentare teoretică, proiectare de detaliu și implementare, testare și validare, manual de instalare și utilizare, concluzii, bibliografie și anexe.*
3. **Locul documentării:** Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare
4. **Consultanți:**
5. **Data emiterii temei:** 1 Octombrie 2025
6. **Data predării:** 10 Iulie 2026

Absolvent: _____

Coordonator științific: _____

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE**
DEPARTAMENTUL CALCULATOARE**Declarație pe propria răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a) _____, _____ legitimat(ă) cu

_____ seria _____ nr. _____
(conform copiei anexate prezentei declarații, copie sem-
nată și certificată "conform cu originalul"), autorul lucrării

elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Programul de studiu _____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____ a anului universitar _____, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administra- tive, respectiv, *anularea examenului de licență*. Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine și conform copiei documentului de identitate menționat mai sus.

Data

Nume, Prenume

Semnătura

Instrucțiuni generale.

De citit înainte (această pagină se va elimina din versiunea finală):

1. Cele trei pagini anterioare (foaie de capăt, foaie sumar, declarație) se vor lista pe foi separate (nu față-verso), fiind incluse în lucrarea listată. Foaia de sumar (a doua) necesită semnătura absolventului, respectiv a coordonatorului. Pe declarație se trece data când se predă lucrarea la secretarii de comisie.
2. Pe foaia de capăt, se va trece corect titulatura cadrului didactic îndrumător, în engleză (consultați pagina de unde ați descărcat acest document pentru lista cadrelor didactice cu titlaturile lor).
3. Fiecare capitol începe pe pagină nouă.
4. Marginile paginilor nu se modifică.
5. Respectați restul instrucțiunilor din fiecare capitol.
6. Am inclus pachetul `hyperref` pentru a genera legături de navigare atât în document cât și la link-uri de web. Pentru listarea pe hârtie a fișierului pdf de comentarii linia care conține `%\hypersetup{hidelinks}` aflată în partea de început a fișierului principal `thesis_rom.tex`.

Cuprins

Capitolul 1	Introducere	1
1.1	Context și motivație	1
1.2	Domeniul și delimitarea temei	1
1.3	Soluția propusă și contribuții	2
1.4	Arhitectură colaborativă	2
Capitolul 2	Obiectivele proiectului	3
2.1	Introducere	3
2.2	Obiectivul general	3
2.3	Obiective principale	3
2.4	Obiective secundare	4
2.5	Cerințe funcționale și non-funcționale	5
2.5.1	Cerințe funcționale	5
2.5.2	Cerințe non-funcționale	5
Capitolul 3	Studiu bibliografic	6
3.1	Instrumente de adnotare semantică	6
3.2	Seturi de date și benchmark-uri	6
3.3	Arhitecturi de segmentare	7
3.4	Modele vizual-lingvistice	8
3.5	Tehnici de adnotare asistată	10
3.6	Evoluția pattern-urilor UI/UX în adnotare	12
3.7	Sincronizare colaborativă	12
3.8	Calitatea datelor și trasabilitate	13
3.9	Scalabilitate și confidențialitate	14
Capitolul 4	Analiză și fundamentare teoretică	16
4.1	Modelul conceptual al soluției	16
4.2	Algoritmi utilizați	16
4.2.1	Algoritmul AutoSeg	16
4.2.2	Algoritmul AutoMask	17
4.2.3	Algoritmul de asistență LLM	17
4.2.4	Algoritmul de sincronizare colaborativă	17
4.3	Modele de inteligență artificială folosite	18
4.3.1	YOLO pentru AutoSeg	18
4.3.2	Mask2Former pentru AutoMask	19
4.3.3	Modele lingvistice bazate pe Transformer	19
4.4	Protocoale utilizate	20
4.4.1	HTTP/REST	20
4.4.2	WebSocket și STOMP	21
4.4.3	SockJS ca fallback	22
4.4.4	AMQP și RabbitMQ	22
4.5	Frameworkuri și platforme	22
4.5.1	Qt și PyQt6 pentru clientul desktop	23
4.5.2	PyTorch pentru modelele externe	23

4.5.3	Java și Spring Boot pentru serviciile backend	23
4.5.4	Docker și orchestrarea serviciilor	24
4.6	Modele abstracte	24
4.6.1	Modelul de acces	24
4.6.2	Modelul de colaborare	24
4.6.3	Modelul de adnotare	25
4.6.4	Modelul de stocare	25
4.7	Structura logică și funcțională a aplicației	25
4.7.1	Clientul desktop	25
4.7.2	Worker-ele locale	26
4.7.3	Serviciile backend	26
4.7.4	Persistență și export	26
4.7.5	Flux funcțional complet	27
4.8	Justificarea integrării modelelor	27
Capitolul 5	Proiectare de detaliu și implementare	28
5.1	Arhitectura generală a sistemului	28
5.2	Structura clientului desktop PyQt	30
5.2.1	Rolul clientului desktop în sistem	30
5.2.2	MainWindow ca orchestrator al aplicației	32
5.2.3	Panourile aplicației	33
5.2.4	Canvasul de adnotare	36
5.2.5	Obiectele grafice de adnotare	37
5.2.6	Integrarea AutoSeg YOLO și AutoMask în client	38
5.2.7	Worker-e și procese externe	41
5.2.8	Ferestre auxiliare: chat, colaborare și chatbot local	42
5.2.9	Gestionarea stării locale	43
5.3	Autentificare și model de acces	45
5.4	Chat în timp real și prezență	46
5.5	Colaborarea pe adnotări	48
5.5.1	Trimiterea adnotărilor mari în mai multe pachete	50
5.6	Serviciile backend și infrastructura Docker	51
5.7	Persistența și structura datelor	53
5.8	Rulare și integrarea componentelor	54
Capitolul 6	Testare și validare	56
Capitolul 7	Manual de instalare și utilizare	57
Capitolul 8	Concluzii	58
Bibliografie		59
Anexa A	Sursele și proveniența figurilor	62

Capitolul 1. Introducere

1.1. Context și motivație

În ultimii ani, aplicațiile de viziune artificială au devenit uzuale în industrie și cercetare, iar calitatea lor depinde direct de seturile de date etichetate. Procesul de adnotare vizuală implică trasarea manuală a obiectelor în imagini, fie prin dreptunghiuri de încadrare, fie prin poligoane sau măști. Pentru seturi mari de date, această muncă este consumatoare de timp și necesită atenție constantă pentru a menține consistența între adnotări. În practică, multe proiecte întârzie sau cresc în cost tocmai din cauza efortului de etichetare.

Motivația lucrării pornește din nevoia de a reduce timpul și efortul necesare adnotării, fără a pierde controlul uman asupra rezultatului final. O abordare semi-automată, în care un model de inteligență artificială propune contururi sau măști, poate accelera semnificativ procesul. Utilizatorul rămâne totuși responsabil de validare și corectare, ceea ce păstrează calitatea datelor. În plus, exportul adnotărilor în formate standard ajută la integrarea rapidă a datelor în fluxuri de antrenare.

Un alt aspect important este colaborarea între mai mulți utilizatori. În proiectele reale, adnotarea se face de obicei în echipă, iar lipsa sincronizării duce la duplicarea muncii și la inconsistențe. Printr-un mecanism de lucru colaborativ, cu actualizări în timp real și gestionarea sesiunilor, echipa poate lucra pe același proiect în mod coordonat. Astfel, se obține un proces mai rapid, mai controlat și mai ușor de urmărit.

1.2. Domeniul și delimitarea temei

Lucrarea se încadrează în domeniul adnotării vizuale pentru date de tip imagine, cu accent pe segmentare și detecție de obiecte. Sunt vizate adnotări realizate prin dreptunghiuri, poligoane și măști, deoarece acestea acoperă cele mai folosite reprezentări în seturile de date moderne.

Domeniul ales include și administrarea etichetelor și a adnotărilor, astfel încât acestea să poată fi organizate și verificate ușor.

Datele rezultate sunt salvate într-un format intern și pot fi exportate în formate standard, precum COCO (Common Objects in Context) și YOLO (You Only Look Once). Astfel, seturile de date pot fi folosite ușor în fluxuri de antrenare, fără pași suplimentari de conversie. Structura de export este gândită pentru a păstra separarea între tipurile de adnotări și pentru a face reutilizarea cât mai simplă.

Tema este delimitată la un instrument desktop de adnotare, care integrează metode semi-automate prin apelarea unor modele externe. Modelul nu este antrenat în cadrul lucrării, ci este folosit ca suport pentru propunerea conturilor. Pentru segmentare asistată este folosit un model de tip Mask2Former, iar pentru detecție și segmentare completă se folosește un model de tip YOLOE.

Nu sunt urmărite subiecte precum optimizarea arhitecturilor de rețele neuronale, compararea extensivă a modelelor sau dezvoltarea unui serviciu cloud pentru adnotare. Accentul este pe realizarea unui instrument practic, ușor de folosit, local, care reduce efortul de etichetare și permite colaborarea eficientă între membri ai aceleiași echipe.

1.3. Soluția propusă și contribuții

Soluția propusă este o aplicație desktop dezvoltată în PyQt6, care oferă un flux de lucru complet pentru adnotarea imaginilor. Utilizatorul poate deschide un folder de imagini, alege rapid fișierele din listă și edita adnotările direct pe canvas. Interfața este organizată în panouri funcționale, astfel încât instrumentele de adnotare, lista de elemente și proprietățile acestora să fie accesibile fără a întrerupe procesul de lucru.

Canvasul este componenta centrală a interfeței. Acesta permite zoom și pan, desenarea de dreptunghiuri, poligoane sau măști, precum și selecția și modificarea elementelor existente. În plus, sunt disponibile ajustări vizuale ale imaginii pentru a crește claritatea în zonele dificil de adnotat. Acest mod de lucru ajută la păstrarea preciziei și reduce efortul în sarcinile repetitive.

Gestionarea adnotărilor se face printr-un manager central, care păstrează toate elementele asociate imaginii curente și oferă operații de adăugare, modificare sau ștergere. Managerul emite notificări către interfață atunci când apar schimbări, ceea ce asigură sincronizarea dintre starea internă și elementele afișate. Fiecare adnotare are un identificator propriu, fapt util pentru editare, export și colaborare.

Aplicația include facilități de tip *quality of life*, menite să reducă erorile și timpul pierdut. La închiderea aplicației sau la schimbarea imaginii, utilizatorul este întrebat dacă dorește să salveze modificările. În plus, aplicația include chat în timp real și suport de tip chatbot local pentru asistență.

Componenta de inteligență artificială este integrată prin AutoSeg și AutoMask, care rulează modele externe ca subprocese asincrone. Rezultatele sunt transformate în adnotări editabile, iar utilizatorul le poate corecta imediat. Astfel, aplicația combină propunerea automată cu controlul manual, obținând un echilibru între viteză și acuratețe.

Contribuțiile principale constau în integrarea unui flux complet de adnotare cu suport pentru segmentare semi-automată, organizarea clară a interfeței și mecanismele de sincronizare în colaborare. Rezultatul este un instrument practic, adaptat nevoilor de etichetare din proiecte reale.

1.4. Arhitectură colaborativă

Backend-ul aplicației este separat pe servicii dedicate pentru autentificare, chat și colaborare pe adnotări. Un reverse proxy direcționează cererile către serviciul corespunzător, iar mesageria facilitează schimbul de informații între module. Această separare permite dezvoltarea și testarea componentelor în mod independent, păstrând totuși un flux unitar pentru utilizator.

Sincronizarea se bazează pe două tipuri de comunicație. Prin REST, clientul listează sesiunile, obține un snapshot al stării curente și poate încărca sau descărca imagini temporare. Prin WebSocket, clienții primesc evenimente în timp real, precum crearea, actualizarea sau ștergerea adnotărilor. Când o imagine este actualizată într-o sesiune, ceilalți utilizatori primesc un eveniment de tip „image available” și sincronizează automat datele.

Modelul de colaborare este centrat pe sesiuni, fiecare sesiune având o stare curentă și o versiune a adnotărilor. Astfel, modificările sunt propagate rapid, iar utilizatorii văd același conținut fără conflicte majore. Autentificarea prin JWT (JSON Web Token) limitează accesul doar la utilizatorii autorizați, iar token-ul este salvat local pentru a evita relogarea repetată.

Capitolul 2. Obiectivele proiectului

2.1. Introducere

Acest capitol prezintă obiectivele proiectului. Ele sunt legate direct de problema adnotării vizuale și descriu ce se urmărește prin realizarea aplicației. Structura include obiectivul general, obiectivele principale și secundare, precum și cerințele funcționale și non-funcționale.

2.2. Obiectivul general

În mod obișnuit, adnotarea datelor vizuale este o muncă manuală, costisitoare și lentă, iar rezultatele pot fi influențate de oboseală și erori umane. Acest context creează nevoia unei soluții care să accelereze procesul, fără a pierde controlul asupra calității.

Obiectivul general al proiectului este dezvoltarea unei aplicații desktop de adnotare vizuală, semi-automată și colaborativă, care reduce timpul de etichetare și păstrează controlul utilizatorului asupra rezultatului final. Aplicația trebuie să ofere un flux de lucru complet, de la încărcarea imaginilor și navigarea rapidă între ele, până la editarea adnotărilor și exportul în formate standard. În mod explicit, trebuie să suporte adnotări de tip bounding box (dreptunghi), poligon și mască (image segmentation), pentru a acoperi scenariile uzuale de detecție și segmentare.

Gestionarea adnotărilor trebuie să fie clară și previzibilă, cu posibilitatea de selecție, editare și ștergere, astfel încât utilizatorul să poată corecta rapid erorile. Rezultatele trebuie să poată fi salvate și exportate în formate standard, pentru a fi folosite ușor în antrenarea modelelor. Prin integrarea unor modele externe pentru propuneri de segmentare și prin suportul pentru colaborare securizată între utilizatori, soluția urmărește să crească eficiența fără a compromite calitatea datelor.

2.3. Obiective principale

Obiectivele principale urmăresc să definească funcționalitățile cheie ale aplicației și modul în care acestea răspund cerințelor de adnotare. Fiecare obiectiv este formulat astfel încât să poată fi implementat și verificat concret.

- **Interfață de adnotare completă.** Realizarea unui canvas interactiv cu zoom și pan, care permite desenarea de adnotări de tip bounding box, poligon și mască. Interfața trebuie să susțină navigarea rapidă între imagini și un control vizual clar.
- **Gestionarea coerentă a adnotărilor.** Păstrarea unei structuri clare pentru etichete și elemente, cu identificatori unici, listare și operații de editare sau ștergere. Această structură reduce erorile și facilitează modificările ulterioare.
- **Export standardizat al datelor.** Oferirea de export în formatele COCO și YOLO, cu organizare predictibilă a fișierelor. Astfel, dataset-urile pot fi folosite direct în antrenare, fără conversii suplimentare.
- **Integrare AI semi-automată.** Folosirea AutoSeg și AutoMask pentru a genera propuneri de segmentare care pot fi corectate imediat de utilizator. Scopul este accelerarea procesului fără a compromite calitatea adnotărilor.

- **Modularizarea execuției AI.** Rularea modelelor ca aplicații separate, prin procese dedicate, pentru a izola erorile și a permite actualizări independente. Această abordare crește stabilitatea și flexibilitatea sistemului.
- **Colaborare în timp real.** Realizarea unui server separat pentru colaborare, cu suport de conectare și sesiuni comune, apoi sincronizare a evenimentelor de tip create, update și delete. Actualizarea imaginii între utilizatori se face prin REST și WebSocket pentru a evita duplicarea muncii.
- **Comunicare între utilizatori.** Oferirea unei interfețe de chat care permite schimbul de mesaje între utilizatori în timpul sesiunilor de lucru.
- **Securitate și acces controlat.** Autentificarea pe bază de JWT și păstrarea locală a token-ului pentru o experiență de lucru fluidă. Accesul la sesiuni și date trebuie să fie controlat și verificabil.
- **Evaluarea și selecția modelelor AI externe.** Testarea Mask2Former și YOLOE pe imagini de referință, cu măsurarea timpului de procesare și a calității rezultatelor. Alegerea finală trebuie justificată pe baza acestor rezultate.
- **Impachetarea și integrarea backend-urilor de inferență.** Folosirea unui script care facilitează build-uirea executabilului, pentru un export simplu și repetabil. Integrarea trebuie să permită conectarea ușoară a backend-urilor la aplicație.

2.4. Obiective secundare

Obiectivele secundare completează funcționalitățile principale prin îmbunătățirea experienței de utilizare și prin validări suplimentare ale componentelor AI și de colaborare.

- **Benchmarking pentru modele alternative.** Analizarea unor modele open-vocabulary (YOLO-World, LLMDet, SAM2) pentru a observa potențialul de generalizare și a justifica alegerea finală.
- **Validarea suportului local de tip VLM/LLM.** Compararea mai multor modele rulate local (Ollama) pentru a susține componenta de chatbot și a stabili un compromis calitate-viteză. Chatbotul are rolul de a ajuta utilizatorul cu întrebări și de a oferi sugestii pentru remedieri ale erorilor întâlnite.
- **Experiență de utilizare și prevenirea erorilor.** Confirmare la schimbarea imaginii și la închidere pentru a evita pierderea adnotărilor.
- **Persistența setărilor vizuale.** Salvarea preferințelor (grosime contur, dimensiune font, opacitate mască) între sesiuni.
- **Îmbunătățirea vizuală a imaginii.** Ajustări de luminozitate, contrast și gamma fără a altera datele originale.
- **Încărcare inteligentă a adnotărilor.** Asocierea automată a fișierelor JSON cu imaginea corespunzătoare.
- **Optimizarea serializării măștilor.** Compresie și reducerea volumului de date pentru fișiere mari.
- **Reducerea traficului în colaborare.** Împărțirea actualizărilor mari (ex. măști) în batch-uri și trimiterea lor prin WebSocket, pentru a evita închiderea prematură a conexiunii. Se aplică și deduplicarea evenimentelor.
- **Crosshair și ghidaj vizual.** Indicatori pentru precizie în adnotare și selectare.

2.5. Cerințe funcționale și non-funcționale

Pentru a susține obiectivele prezentate, proiectul are un set de cerințe funcționale și non-funcționale, sintetizate în tabelele de mai jos.

2.5.1. Cerințe funcționale

Cerință	Descriere scurtă
Încărcare și navigare imagini	Deschidere folder, listare rapidă și comutare între imagini.
Canvas de adnotare	Zoom, pan și trasare de bounding box, poligon și mască.
Editare adnotări	Selectare, modificare, ștergere, etichete și culori ușor de gestionat.
Import/Export	Salvare JSON și export COCO/YOLO cu structură predictibilă.
AI semi-automată	AutoSeg/AutoMask pentru propuneri rapide, corectabile imediat.
Modularizare AI	Rulare modele ca procese separate pentru izolare și actualizări.
Colaborare	Sesiuni, sincronizare create/update/delete prin REST și WebSocket.
Autentificare	Acces controlat cu JWT și token salvat local.
Comunicare	Chat între utilizatori și chatbot local pentru întrebări și erori.
Personalizare	Setări vizuale persistente și confirmare la închidere/schimbare.

2.5.2. Cerințe non-funcționale

Cerință	Descriere scurtă
Usabilitate	Interfață clară, operații frecvente accesibile rapid.
Performanță	Interfață responsivă în timpul procesării AI (asincron).
Fiabilitate	Validări pentru căi inexistente și mesaje clare de eroare.
Securitate	Protecția sesiunilor și a datelor prin token.
Integritate date	Exporturi consistente, fără pierderi de informație.
Scalabilitate	Mai mulți utilizatori pot lucra în aceeași sesiune.
Mentenabilitate	Arhitectură modulară pentru întreținere și extindere.
Extensibilitate	Adăugare de modele și formate noi fără refactor major.
Portabilitate	Rulare locală pe Windows cu configurare minimă.

Capitolul 3. Studiu bibliografic

Studiul bibliografic este organizat progresiv: de la instrumente și seturi de date existente, spre arhitecturi de segmentare și modele vizual-lingvistice, continuând cu tehnici de adnotare asistată și pattern-uri de interfață, și încheind cu sincronizare colaborativă, calitate și confidențialitate.

3.1. Instrumente de adnotare semantică

Instrumentele de adnotare reprezintă interfața dintre datele brute și seturile de date structurate necesare antrenării modelelor. Calitatea și eficiența acestor instrumente influențează direct volumul și consistența adnotărilor produse, cu efecte directe asupra performanței modelelor antrenate ulterior.

În raportul tehnic din 2017 al lui Breitenfeld și Muller-Birn se realizează o comparație sistematică a instrumentelor de adnotare semantică utilizate în contexte de cercetare, cu accent pe legarea conținutului la vocabulare controlate și baze de cunoștințe [1]. Autorii analizează uneltele după mai multe criterii: tipul de utilizator vizat, formatele de fișiere suportate, sursele de cunoștințe (RDF/SKOS), modul de interacțiune cu utilizatorul și contextul de rulare (aplicație web, desktop sau integrată în infrastructuri de cercetare). Analiza evidențiază diferențe clare între uneltele orientate spre utilizatori tehnici, care oferă mai multă flexibilitate, și cele care încearcă să includă experți de domeniu prin interfețe mai accesibile.

Concluzia principală este că majoritatea instrumentelor existente sunt construite cu utilizatorul tehnic în minte, iar suportul pentru colaborare în echipe și pentru trasabilitatea modificărilor rămâne limitat sau absent. Această lipsă îngreunează adopția în echipe interdisciplinare, unde adnotatorii au niveluri diferite de expertiză tehnică și unde istoricul deciziilor de adnotare trebuie să poată fi auditat. Deși lucrarea vizează adnotarea semantică și nu pe cea vizuală, observațiile privind uzabilitatea, colaborarea și trasabilitatea sunt direct relevante pentru proiectul de față, în care aceleași cerințe apar în procesul de etichetare a imaginilor. Din această perspectivă, studiul furnizează argumente solide pentru alegerea unui flux de lucru simplificat, orientat pe utilizator și cu suport explicit pentru colaborare.

3.2. Seturi de date și benchmark-uri

Seturile de date publice au jucat un rol central în standardizarea evaluării și în accelerarea progresului din domeniu. Ele definesc nu doar datele de antrenare și testare, ci și convențiile de adnotare, metricile de evaluare și nivelul de granularitate așteptat de la un instrument de etichetare.

PASCAL Visual Object Classes (VOC) reprezintă unul dintre primele benchmark-uri utilizate pe scară largă pentru detecție și segmentare [2]. Setul de date include etichete pentru 20 de clase comune de obiecte, oferind bounding box-uri pentru sarcina de detecție și măști de segmentare pentru edițiile care le includ. Competițiile anuale organizate în jurul PASCAL VOC au consolidat metrici standard, precum Average Precision (AP) calculat pe curba precizie-recall pentru detecție și Intersection over Union (IoU)/mean IoU

pentru segmentare. Aceste competiții au furnizat un cadru consistent de comparare a metodelor, de la descriptorii HOG cu clasificatoare SVM până la arhitecturile timpurii bazate pe rețele convoluționale profunde, precum R-CNN și variante. Importanța PASCAL VOC constă nu doar în datele în sine, ci în faptul că a impus practici de adnotare și protocoale de evaluare pe care comunitatea le-a urmat pe termen lung. Rămâne relevant ca reper metodologic și punct de referință în analiza evoluției practicilor de adnotare.

Microsoft COCO extinde semnificativ PASCAL VOC atât ca dimensiune, cât și ca complexitate, propunând un set de date de referință pentru detecție, segmentare și descriere de imagini în scene cotidiene cu obiecte multiple și parțial ocluzionate [3]. Colecția include peste 300.000 de imagini și mai mult de două milioane de instanțe adnotate, organizate pe 80 de clase obiect grupate în supercategorii. Anotările sunt detaliate și acoperă mai multe niveluri: segmentare pe instanță prin poligoane, bounding box-uri și descrieri textuale libere ale scenei. Procesul de adnotare a necesitat un efort considerabil, estimat la zece ori mai mare per imagine față de PASCAL VOC, ceea ce reflectă costul real al adnotărilor de calitate. Evaluarea utilizează metrici de precizie medie calculate la multiple praguri IoU (de la 0.5 la 0.95), pentru a reflecta mai realist calitatea localizării și nu doar detectarea obiectelor. COCO a standardizat protocoalele de evaluare și a devenit referința principală pentru compararea modelelor, influențând direct modul în care sunt concepute instrumentele moderne de adnotare.

Torrallba și Efros analizează bias-ul seturilor de date din recunoașterea vizuală și arată că modelele învață adesea caracteristici specifice colecțiilor de antrenare, nu concepte vizuale generale [4]. Autorii compară mai multe seturi de date cunoscute, PASCAL VOC, ImageNet, LabelMe, SUN09, Caltech101, MSRC, prin două tipuri de experimente: testul „Name That Dataset”, în care un clasificator trebuie să identifice colecția de origine a unei imagini, și teste de generalizare încrucișată, unde modelele antrenate pe o colecție sunt evaluate pe alta. Rezultatele sunt revelatoare: acuratețea scade semnificativ atunci când antrenarea și testarea se realizează pe surse diferite, iar bias-ul de selecție (ce imagini sunt incluse), bias-ul de captură (unghiuri, iluminare tipice) și bias-ul exemplarelor negative (ce obiecte apar în fundal) afectează direct robustețea modelelor în condiții reale. Deși studiul precede era rețelelor profunde și se bazează pe metode HOG/BoW cu SVM, concluziile privind bias-ul rămân valabile și în contextul arhitecturilor moderne, care pot amplifica aceste artefacte de distribuție. Pentru un instrument de adnotare colaborativă, lucrarea susține importanța fluxurilor de asigurare a calității, a consensului între adnotatori și a controlului activ asupra distribuției claselor și a exemplarelor negative în seturile construite.

3.3. Arhitecturi de segmentare

Arhitecturile de segmentare sunt componenta centrală a oricărui sistem de adnotare asistată. Evoluția lor, de la rețele complet convoluționale la arhitecturi bazate pe mecanisme de atenție, a permis creșterea preciziei și extinderea domeniului de aplicare, de la segmentare semantică simplă până la înțelegerea panoptică a scenei.

U-Net propune o arhitectură complet convoluțională pentru segmentare semantică, concepută inițial pentru imagini biomedicale în condiții cu puține date adnotate [5]. Structura în formă de U combină o cale de contractare (encoder) pentru extragerea caracteristicilor la rezoluții din ce în ce mai mici cu una de expansiune (decoder) pentru reconstrucția rezoluției originale, legate prin conexiuni de tip skip care transmit direct hărțile de caracteristici de la encoder la decoder. Aceste conexiuni skip sunt esențiale pentru păstrarea detaliilor spațiale fine, care s-ar pierde altfel în procesul de compresie.

Modelul utilizează o strategie overlap-tile pentru procesarea imaginilor de dimensiuni mari fără a fi limitat de memoria GPU, augmentare cu deformări elastice pentru a crește variabilitatea datelor de antrenare și o funcție de pierdere ponderată care accentuează granițele dintre obiectele adiacente. Evaluările pe datele ISBI de segmentare celulară au confirmat eficiența abordării chiar și cu seturi de antrenare de câteva zeci de imagini. U-Net a devenit un punct de referință în segmentarea semantică și a influențat numeroase arhitecturi ulterioare. Pentru proiect, modelul susține ideea de auto-segmentare rapidă în interfață, antrenare eficientă pe seturi mici create colaborativ și procesare pe dale pentru imagini de rezoluție mare.

Hariharan et al. propun un cadru unificat pentru detecție și segmentare la nivel de instanță, care prezice simultan bounding box-uri și măști pentru fiecare obiect din imagine [6]. Metoda extinde detectoarele bazate pe propuneri de regiuni prin adăugarea unui modul de predicție a măștii per instanță, oferind o înțelegere mai fină a scenei față de segmentarea semantică clasică, care nu diferențiază obiectele individuale din aceeași clasă. Relevanța pentru proiect este directă: adnotarea de instanțe presupune atât localizarea precisă a obiectelor, cât și delimitarea completă a conturilor lor, ceea ce justifică arhitecturi de instrumente care combină desenarea de bounding box-uri cu rafinarea ulterioară a măștilor de segmentare.

Kirillov et al. unifică segmentarea semantică și cea de instanțe într-o sarcină unică numită segmentare panoptică, în care fiecare pixel din imagine primește atât o categorie semantică (ex. drum, cer, mașină), cât și, pentru obiectele numărabile, un identificator de instanță unic [7]. Autorii introduc metrica Panoptic Quality (PQ), calculată ca produs între calitatea segmentării și calitatea recunoașterii, oferind un criteriu de evaluare unificat pentru ambele tipuri de regiuni. Formularea panoptică a impus cerințe mai stricte asupra adnotărilor, deoarece fiecare pixel trebuie atribuit unambiguu, fără zone neadnotate. Lucrarea a influențat arhitecturi ulterioare și este relevantă pentru proiect prin cerința de adnotări complete și consistente la nivel de imagine, fără suprapuneri sau lacune.

Mask2Former propune o arhitectură universală pentru segmentare, bazată pe clasificarea măștilor și pe mecanismul de object queries, care poate aborda simultan segmentarea semantică, de instanțe și panoptică folosind aceeași rețea [8]. Inovația principală este mecanismul de masked attention: spre deosebire de cross-attention global, fiecare query poate accesa doar pixelii din regiunea prezisă în iterația anterioară, reducând zgomotul de fundal și accelerând convergența. Decodorul rulează runde succesive de atenție pe hărți de caracteristici la rezoluții diferite ($1/32$, $1/16$, $1/8$), permițând modelului să rafineze progresiv predicțiile. Pentru a reduce consumul de memorie la antrenare, pierderile nu sunt calculate pe toți pixelii, ci pe un eșantion de puncte informative selectate adaptiv. Rezultatele arată performanță SOTA simultană pe COCO panoptic (PQ), COCO instance (AP) și ADE20K semantic (mIoU), cu convergență în aproximativ 50 de epoci față de sute de epoci la arhitecturi precedente. Limitările includ nevoia de antrenare separată per sarcină sau set de date și o performanță mai slabă pe obiecte foarte mici. Pentru proiect, Mask2Former este relevant prin posibilitatea de fine-tuning eficient pe seturi create colaborativ și prin generarea de propuneri cu focalizare locală, utilă pentru uneltele de pre-selectare și corecție rapidă.

3.4. Modele vizual-lingvistice

Modelele vizual-lingvistice au schimbat paradigma adnotării prin eliminarea necesității de a defini în avans un vocabular fix de clase. Capacitatea de a înțelege descrieri textuale libere și de a le lega de regiuni vizuale deschide noi posibilități pentru pre-adnotare

automată și interogare flexibilă a datelor.

CLIP introduce o metodă de învățare a reprezentărilor vizual-lingvistice prin antrenarea simultană a unui encoder vizual și a unui textual pe perechi imagine-text extrase de pe internet la scară de sute de milioane de exemple [9]. Obiectivul contrastiv maximizează similaritatea cosinusului pentru perechile imagine-text corecte și o minimizează pentru cele incorecte din cadrul aceluiași batch. Această abordare permite clasificare zero-shot: etichetele țintă sunt formulate ca prompturi textuale de tipul „A photo of a {label}.”, iar imaginea este clasată prin potrivire cu descrierea cea mai probabilă, fără a fi nevoie de exemple de antrenare specifice acelei sarcini. CLIP se dovedește competitiv cu modele supervizate pe ImageNet fără antrenare specifică și mai robust la schimbări de distribuție față de modelele clasice. Encoderul vizual a devenit un fundament pentru modelele de segmentare cu vocabular deschis și pentru sistemele multimodale care raționează simultan pe text și imagini. Pentru proiect, CLIP este relevant ca bază pentru interogări textuale flexibile în pre-adnotare și pentru sugestii rapide fără antrenare suplimentară, dar necesită verificare umană și control al bias-urilor transferate din datele web.

Liang et al. abordează o limitare specifică a CLIP la segmentare: performanța modelului scade semnificativ pe imagini mascate, din cauza diferenței de domeniu față de datele de pre-antrenare și a tokenilor de fundal neinformativi introduși de mascare [10]. Soluția propusă adaptează CLIP pe regiuni mascate prin construirea unui set de date zgomotos din subtitrările COCO și prin tehnica Mask Prompt Tuning (MPT), care înlocuiește tokenii de fundal mascați cu tokeni de prompt antrenabili, fie cu CLIP înghețat, fie cu fine-tuning complet. Modelul rezultat, OVSeg, obține rezultate SOTA în regim zero-shot pe mai multe benchmark-uri de segmentare cu vocabular deschis, incluzând ADE20K-150, ADE20K-847 și Pascal Context-459. Limitările includ influența claselor văzute la antrenare asupra propunerilor de măști și decalajul de performanță dintre clasele văzute și cele nevăzute. Pentru proiect, tehnica susține interogări textuale libere în adnotare, pre-adnotare automată din metadate textuale și adaptare eficientă cu resurse de calcul reduse, datorită numărului mic de parametri antrenabili din MPT.

YOLOE propune un model unificat, open-set și în timp real pentru detecție și segmentare, integrând prompturi textuale, vizuale și un mod fără prompt în aceeași arhitectură [11]. Pentru prompturi textuale, blocul RepRTA rafinează embedding-urile textuale la antrenare printr-o rețea de tip FFN, apoi re-parametrizează această rețea ca o convoluție 1×1 în capul de clasificare, eliminând complet costul suplimentar la inferență. Pentru prompturi vizuale, blocul SAVPE separă o ramură semantică (caracteristici agnostice față de prompt) de o ramură de activare (ponderi specifice promptului), agregându-le eficient pentru a produce un embedding vizual reprezentativ. În modul fără prompt, LRPC localizează toate obiectele din imagine fără un vocabular explicit, asociind ancorele relevante cu un vocabular extins doar la momentul inferenței. Rezultatele arată o reducere de circa trei ori a costului de antrenare față de YOLO-World, câștiguri consistente pe LVIS în regim zero-shot și viteză mai mare la inferență. Limitările includ dependența de un vocabular static în modul fără prompt și necesitatea generării de pseudo-măști cu SAM pentru antrenarea componentei de segmentare. Pentru proiect, YOLOE este relevant prin suportul multimodal, posibilitatea unui mod de auto-etichetare eficient și eliminarea costurilor suplimentare la rulare prin re-parametrizare, ceea ce facilitează deployment pe servere cu resurse limitate.

LLaVA propune o arhitectură care îmbină un encoder vizual (CLIP ViT-L/14) cu un model lingvistic de mari dimensiuni (Vicuna) printr-un strat de proiecție liniar, permițând dialogul vizual-textual detaliat bazat pe instrucțiuni naturale [12]. Autorii re-

zolvă lipsa datelor de antrenare de tip dialog vizual prin generarea automată de conversații om-AI condiționate de imagini cu ajutorul GPT-4, folosind doar metadatele imaginilor (subtitrări și bounding box-uri) ca input textual pentru GPT-4, fără a-i furniza imaginile direct. Modelul antrenat astfel atinge aproximativ 85.1% din performanța GPT-4 pe sarcini vizuale complexe și obține rezultate competitive pe benchmark-ul ScienceQA, care necesită raționament vizual-textual multi-pas. Limitările includ tendința la halucinații, inventarea de detalii care nu sunt prezente în imagine, și dificultăți la citirea textului mic sau a elementelor subtile de interfață, cauzate de limitările rezoluției encoderului vizual. Pentru proiect, LLaVA este relevant ca mecanism de asistență inteligentă: la trimiterea unei capturi de ecran din interfața de adnotare, modelul poate identifica elementele vizuale și răspunde la întrebări de tipul „Ce reprezintă zona selectată?” sau „De ce a fost propusă această etichetă?”, fără a efectua acțiuni automate în sistem.

DistilBERT reduce dimensiunea modelului BERT cu 40% prin distilarea cunoștințelor direct în faza de pre-antrenare, folosind o funcție de pierdere compusă din trei componente: distilare (imitarea distribuțiilor soft ale modelului profesor), modelare a limbajului (obiectivul clasic MLM) și aliniere a embedding-urilor intermediare prin similaritate cosinusă [13]. Rezultatul este un model mai rapid, cu un consum de memorie semnificativ redus, care păstrează aproximativ 97% din capacitatea de înțelegere a limbajului pe benchmark-ul GLUE. Principiul de distilare poate fi aplicat și altor modele lingvistice mari, oferind o cale generală spre modele mai eficiente. Pentru proiect, DistilBERT sau modele similare distilate sunt relevante pentru rularea locală a sarcinilor de procesare a limbajului natural, precum clasificarea intențiilor utilizatorului, completarea automată a etichetelor sau sugerarea categoriilor din vocabularul proiectului, reducând latența și costurile față de apelul unui model extern de dimensiuni mari.

3.5. Tehnici de adnotare asistată

Adnotarea asistată combină inferența automată a modelelor cu verificarea și corecția umană, reducând efortul de etichetare fără a renunța la controlul calității. Această secțiune prezintă principalele tehnici din literatura de specialitate, de la selecția inteligentă a exemplelor de adnotat până la generarea interactivă a măștilor.

Settles descrie în raportul său tehnic principiile și strategiile de active learning, arătând că un model poate obține performanțe ridicate cu un număr mult mai mic de exemple etichetate dacă selectează activ instanțele cele mai informative pentru adnotare [14]. Sunt acoperite trei scenarii principale: sinteza interogărilor (generarea de exemple artificiale), eșantionarea din flux (selectarea din datele care sosesc în timp real) și eșantionarea dintr-un rezervor (selectarea dintr-un pool fix de exemple neetichetate). Strategiile de interogare includ selecția bazată pe incertitudine (exemple aproape de granița de decizie), comitete de modele (exemple cu dezacord maxim între modele) și reducerea erorii sau varianței așteptate. În practică, active learning poate reduce substanțial efortul de etichetare, dar introduce provocări legate de costul variabil al etichetării diferitelor exemple, adnotatori cu erori și bias indus de strategia de selecție, care poate crea un set de antrenare nereprezentativ pentru distribuția reală a datelor. Pentru un instrument colaborativ de adnotare, aceste idei permit prioritizarea imaginilor sau a regiunilor cu cel mai mare aport informațional, evitând munca redundantă pe exemple deja bine reprezentate în model.

Tehnicile Grad-CAM (Gradient-weighted Class Activation Mapping) folosesc gradientul semnalului de pierdere față de activările ultimului strat convoluțional pentru a produce o hartă de căldură a regiunilor care au contribuit cel mai mult la decizia modelu-

lui [15]. Această hartă poate fi suprapusă ca overlay transparent peste imaginea originală, oferind o explicație vizuală intuitivă a predicției. Spre deosebire de metodele anterioare (ex. CAM clasic), Grad-CAM nu necesită modificarea arhitecturii rețelei și poate fi aplicat oricărui model convoluțional standard. Pentru proiect, aceste tehnici sunt utile ca mecanisme de transparență și asigurare a calității: pot explica de ce modelul a sugerat o anumită etichetă sau mască, pot evidenția erori de focalizare (modelul se concentrează pe fundalul tipic clasei, nu pe obiectul propriu-zis) și pot sprijini detectarea timpurie a bias-ului din datele de antrenare. Afișarea hărților ca overlay în interfața colaborativă facilitează validarea rapidă a sugestiilor automate și ghidează adnotatorii spre corectarea regiunilor problematice.

Polygon-RNN generează adnotări de instanțe ca poligoane produse secvențial de un model recurrent (RNN), ale cărui intrări sunt caracteristici vizuale extrase de un CNN aplicat pe decupajul obiectului [16]. Modelul prezice coordonatele vârfurilor pe o grilă discretizată, unul câte unul, și se oprește la un token special de final, transformând trăsarea completă a unui contur într-un proces de verificare și ajustare a câtorva vârfuri propuse. Utilizatorul poate corecta orice vârf greșit în timp real, iar modelul poate regenera secvența de la acel punct. Autorii raportează accelerări semnificative ale timpului de adnotare față de metoda manuală completă, menținând un acord ridicat cu adnotările de referință exprimat prin IoU. Pentru proiect, această abordare susține un flux de asistență AI cu feedback uman în buclă scurtă, în care interfața trebuie să permită corecturi rapide la nivel de vârf și să integreze imediat rezultatul corectat în forma finală fără a reporni întregul proces.

Deep Extreme Cut (DEXTR) reduce efortul de adnotare prin generarea unei măști de segmentare precise pornind de la doar patru puncte extreme ale obiectului, cel mai de sus, cel mai de jos, cel mai din stânga și cel mai din dreapta pixel al conturului [17]. Aceste patru puncte sunt codate ca hărți de densitate Gaussiană și concatenate cu imaginea originală ca al cincilea canal de intrare al rețelei, ghidând modelul să producă o mască focalizată pe zona de interes. Utilizatorul nu mai trasează poligoane complete, ci indică rapid limitele extreme ale obiectului în câteva click-uri, iar modelul produce o mască inițială de calitate comparabilă cu adnotarea manuală detaliată. Limitarea principală apare la obiecte puternic ocluzionate sau cu forme foarte neregulate, unde punctele extreme pot fi ambigue sau insuficiente pentru a delimita corect obiectul. Pentru proiect, DEXTR susține un flux de tipul „indicare sumară, apoi corecție punctuală”: masca inițială poate fi ajustată cu pensula sau prin re poziționarea unuia dintre punctele extreme pentru re-inferență rapidă. Comparativ cu Polygon-RNN, DEXTR reduce și mai mult interacțiunea inițială, dar depinde mai mult de calitatea inferenței modelului.

ScribbleBox propune un flux în două etape pentru adnotarea video de instanțe: componenta Curve-VOT urmărește obiectul de-a lungul secvenței printr-o curbă temporală parametrizată cu câteva puncte de control, iar Scribble-VOS rafinează măștile pe baza scribble-urilor propagate prin intermediul unui Graph Convolutional Network (GCN) [18]. Pe setul de referință DAVIS2017, metoda atinge 88.92% J&F (Jaccard and F-measure) cu circa 9.1 click-uri per secvență și necesită corecții explicite pe doar 4 cadre din întreaga secvență. Autorii furnizează și o analiză detaliată a latenței, relevantă pentru designul sistemului: propagarea măștilor la rezoluție 512px ajunge la circa 295 ms/cadru, la 256px circa 152 ms, iar cu cache la 256px circa 82 ms; propagarea scribble-urilor este de 15–28 ms, iar curve fitting și interacțiunea cu caseta sunt sub 15 ms fiecare. Limitarea principală este dependența de calitatea urmăririi inițiale: obiectele mici, parțial ocluzionate sau cu mișcări imprevizibile necesită mai multe puncte de control și corecții

manuale suplimentare. Pentru un instrument colaborativ, structura în două etape permite împărțirea sarcinilor pe niveluri de expertiză, iar latențele măsurate impun cerințe concrete de design: interfață asincronă cu indicatori de progres vizibili, cache persistent al rezultatelor intermediare și mod rapid (rezoluție mică) versus precis (rezoluție mare) selectabil de utilizator.

3.6. Evoluția pattern-urilor UI/UX în adnotare

Designul interfeței de adnotare influențează direct productivitatea adnotatorilor, consistența rezultatelor și capacitatea sistemului de a integra asistența automată. Evoluția acestor interfețe reflectă atât progresele din computer vision, cât și lecțiile acumulate din studiile de uzabilitate cu adnotatori reali.

Evoluția interfeței de adnotare reflectă trecerea de la interacțiunea manuală completă la cea asistată de modele. LabelMe, unul dintre primele instrumente web de adnotare vizuală la scară largă, permite editarea directă a poligoanelor prin operații de zoom, pan, adăugare și ștergere de noduri [19]. Această abordare oferă precizie ridicată și control deplin al adnotatorului, dar implică un cost de timp semnificativ pentru obiecte cu contururi complexe, mai ales când numărul de noduri al poligonului crește. De asemenea, LabelMe nu oferă mecanisme native de colaborare sau de asigurare a calității, ceea ce limitează utilizarea sa în echipe mari. Instrumente moderne precum ScribbleBox și CVAT au evoluat spre o paradigmă în care utilizatorul furnizează indicii brute (casete de delimitare sau scribble-uri), iar modelul generează rapid măști de calitate cu feedback aproape instant [18, 20]. Modele eficiente precum YOLOE susțin acest feedback în regim open-vocabulary, fără costuri mari de inferență și fără a necesita redefinirea vocabularului de clase [11].

CVAT (Computer Vision Annotation Tool) reprezintă unul dintre cele mai complete instrumente open-source de adnotare, structurând munca în ierarhii de proiecte, sarcini și job-uri, ceea ce reduce conflictele de acces și permite paralelizarea activității între adnotatori [20]. Interfața suportă multiple tipuri de adnotări (bounding box-uri, poligoane, cuboids, tracking video), iar integrarea cu modele de AI permite auto-adnotarea și rafinarea interactivă. Un flux în două etape similar cu ScribbleBox, casete brute generate de adnotatorii juniori, urmate de rafinarea măștilor complexe de către adnotatorii seniori, permite o alocare eficientă a sarcinilor pe niveluri de expertiză. Pattern-urile de tip issues și comentarii, combinate cu corecțiile non-distructive prin scribble-uri, reduc numărul de refaceri complete și accelerează ciclul de verificare. Practici suplimentare, precum consensul între adnotatori și honeypot-urile (sarcini cu adnotări cunoscute introduse pentru evaluarea calității), pot fi integrate pentru a măsura consistența și a detecta erori sistematice fără a întrerupe fluxul principal de lucru.

3.7. Sincronizare colaborativă

Colaborarea în timp real pe același set de adnotări ridică provocări tehnice specifice: mai mulți utilizatori pot modifica simultan aceleași obiecte, iar sistemul trebuie să reconcilieze aceste modificări fără a produce stări inconsistente sau conflicte ireparabile.

Sun et al. analizează diferențele fundamentale dintre OT (Operational Transformation) și CRDT (Commutative Replicated Data Type) printr-un cadru de transformare unificat care permite compararea lor pe baze comune [21]. Autorii introduc o distincție clară între starea vizibilă de utilizator (external state, ceea ce vede utilizatorul pe ecran) și structurile interne utilizate pentru sincronizare (internal state, reprezentările care permit

reconcilierea operațiilor concurente). Analiza arată că și CRDT, deși perceput adesea ca o soluție mai simplă, implică transformări indirecte și costuri suplimentare de memorie și calcul, ceea ce poate afecta performanța în sisteme interactive cu latență mică. Concluziile sugerează că, pentru aplicații cu un server central și colaborare în timp real cu număr moderat de utilizatori, un model operațional bazat pe propagarea și transformarea evenimentelor este mai predictibil, mai ușor de depanat și mai eficient decât un CRDT complex. Pentru un instrument de adnotare colaborativă, această perspectivă susține transmiterea operațiilor atomice (creare/modificare/ștergere de adnotări) în locul trimerii periodice a stării complete, reducând lățimea de bandă necesară și simplificând logica de reconciliere.

Arhitecturile event-driven facilitează colaborarea în timp real prin transmiterea asincronă a evenimentelor peste conexiuni persistente, decuplând producătorii de evenimente de consumatori. STOMP (Simple Text Oriented Messaging Protocol) oferă un protocol simplu, bazat pe text și pe principiul publicare/abonare, proiectat pentru a rula peste TCP sau WebSocket [22]. Protocolul definește un set minimal de frame-uri standard, CONNECT, SEND, SUBSCRIBE, MESSAGE, ACK/NACK, DISCONNECT, și separă explicit canalul de transport de logica de rutare a mesajelor, ceea ce facilitează integrarea cu brokeri de mesaje diferiți (ex. RabbitMQ, ActiveMQ) fără modificări în codul clientului. Simplitatea protocolului îl face accesibil pentru implementare atât în frontend (JavaScript), cât și în backend (Java, Python), asigurând interoperabilitate. Pentru proiect, STOMP susține broadcast instant al modificărilor pe canvas, mecanisme de locking al obiectelor editate (pentru a preveni conflictele), notificări de prezență și sincronizarea comentariilor și a modificărilor de stare ale sarcinilor.

3.8. Calitatea datelor și trasabilitate

Calitatea adnotărilor este un factor determinant pentru performanța modelelor antrenate pe aceste date. Un sistem de adnotare colaborativă trebuie să ofere nu doar instrumentele de etichetare, ci și mecanismele de verificare, audit și trasabilitate care garantează că adnotările sunt corecte, consistente și reproductibile.

Lucrarea „Counting on Consensus” oferă un ghid practic și teoretic pentru alegerea metricilor de acord între adnotatori (Inter-Annotator Agreement, IAA), subliniind că procentul brut de acord este înșelător în clasele dezechilibrate, deoarece nu corectează acordul care ar apărea prin pură coincidență [23]. Lucrarea recomandă metrici corectate pentru hazard, precum Cohen’s Kappa (pentru doi adnotatori) sau Krippendorff’s Alpha (pentru mai mulți adnotatori și diverse scale de măsurare). Pentru etichete continue sau ordonate sunt relevante variante precum Kappa ponderată, care penalizează diferit dezacordurile în funcție de gravitatea lor, sau ICC (Intraclass Correlation Coefficient), utilizat frecvent în evaluările clinice. Pentru adnotări structurate, cum sunt măștile de segmentare, sunt mai potrivite măsuri de suprapunere spațială (F1/Dice) și metrici de frontieră care penalizează erorile de localizare. Deși focusul lucrării este pe NLP, principiile se transferă direct la adnotarea de imagini: sistemul poate calcula IAA pe etichete de clasă și pe suprafețele măștilor, poate semnală automat cazurile cu dezacord ridicat între adnotatori pentru revizie și poate impune raportarea intervalelor de încredere în loc de valori punctuale, sprijinind un pipeline de asigurare a calității bazat pe consistență măsurabilă.

Standardul W3C PROV-DM oferă un model formal și interoperabil pentru descrierea provenienței entităților digitale, adică a modului în care acestea sunt create, transformate și utilizate [24]. Modelul introduce trei concepte fundamentale, entitate (un obiect

digital sau o adnotare), activitate (un proces care transformă sau generează entități) și agent (o persoană sau un sistem care inițiază activități), și relații semantice precum *wasGeneratedBy* (entitate produsă de o activitate), *wasDerivedFrom* (entitate derivată din alta) și *wasAssociatedWith* (activitate asociată unui agent). Aceste relații permit reconstruirea completă a istoricului unei adnotări: cine a creat-o, când, pe baza căror date de intrare, cine a modificat-o ulterior și de ce. În contextul adnotării colaborative cu componente automate, PROV-DM este esențial pentru a distinge între adnotările produse de utilizatori umani și cele generate de modele, pentru auditarea calității și pentru reproducibilitatea experimentelor.

Metadatele structurate ale unui sistem de adnotare, proiecte, sarcini, utilizatori, etichete, istoricul modificărilor, comentarii, sunt potrivite pentru un sistem de gestiune a bazelor de date relaționale (RDBMS) precum PostgreSQL, care oferă tranzacții ACID, control al concurenței multi-utilizator prin MVCC și indexare eficientă pentru interogări complexe [25]. Fișierele media (imagini, videoclipuri) sunt stocate separat în sisteme de fișiere locale sau în object storage distribuit și sunt referențiate prin chei sau URL-uri în baza de date, asigurând o separare clară între datele structurate, supuse unui management strict al consistenței, și cele nestructurate, supuse altor cerințe de scalabilitate. Această arhitectură este standard în sistemele moderne de adnotare și permite scalarea independentă a celor două componente. În proiectul de față, PostgreSQL este utilizat în principal pentru autentificare și managementul sesiunilor, dar rămâne opțiunea optimă pentru extensiile viitoare care vor necesita organizarea metadatelor la o scară mai mare.

3.9. Scalabilitate și confidențialitate

Pe măsură ce sistemele de adnotare colaborativă cresc în dimensiune și în numărul de utilizatori concurenți, apar provocări suplimentare legate de distribuirea eficientă a sarcinilor de calcul și de protecția datelor sensibile ale utilizatorilor.

Neurosurgeon propune o schemă de inteligență colaborativă cloud-edge, în care o rețea neuronală adâncă este partiționată la un punct optim, cu straturile inițiale executate local pe dispozitivul utilizatorului și straturile finale executate pe serverul cloud [26]. Modelul profiler estimează timpul de execuție și consumul de energie pentru fiecare strat al rețelei, atât în execuție locală, cât și în cloud, luând în calcul și costul transferului activărilor intermediare (care depinde de lățimea de bandă disponibilă). Pe baza acestor estimări, sistemul alege dinamic stratul L la care activarea este transmisă către server, optimizând fie latența totală, fie consumul energetic al dispozitivului. Evaluările pe mai multe arhitecturi CNN arată că această partiționare adaptivă reduce semnificativ latența end-to-end față de rularea completă în cloud (eliminând latența de transfer a imaginii brute) și față de rularea completă locală (eliminând costul straturilor profunde). Pentru proiect, această abordare permite rularea locală a straturilor ușoare pentru feedback vizual rapid (proponeri brute de bounding box-uri sau segmentări grosiere), delegând straturile costisitoare de rafinare către backend, cu o experiență interactivă fluidă adaptivă la calitatea conexiunii de rețea a fiecărui utilizator.

Lucrarea „Privacy-Preserving Collaborative Learning” propune o schemă de învățare colaborativă cu protecție diferențiată la nivel de participant, în care fiecare agent poate defini propriul buget de confidențialitate prin adăugarea de zgomot Gaussian calibrat în procesul de agregare a parametrilor modelului [27]. Autorii utilizează cadrul zCDP (zero-Concentrated Differential Privacy) pentru a cuantifica și a compune garanțiile de confidențialitate pe parcursul multiplelor runde de antrenare, oferind garanții formale atât pentru transmisia locală (uplink), cât și pentru modelul global distribuit (down-

link). Analiza de convergență demonstrează că schema atinge o soluție optimă chiar și cu niveluri ridicate de perturbație, cu condiția calibrării corecte a zgomotului. Conceptele din lucrare fundamentează politicile de autentificare și autorizare necesare într-un sistem de adnotare multi-tenant cu date sensibile, unde izolarea strictă între proiectele și utilizatorii diferiți este o cerință de bază. Arhitectura susține și un flux de pre-etichetare pe dispozitiv (on-device), în care serverul central primește doar actualizările de model ofuscate, fără acces direct la imaginile brute ale utilizatorilor, o proprietate importantă în domeniile cu cerințe stricte de confidențialitate, precum medicina sau datele personale.

Capitolul 4. Analiză și fundamentare teoretică

4.1. Modelul conceptual al soluției

Sistemul propus este un instrument de adnotare asistată, în care utilizatorul controlează rezultatul final, iar componentele automate produc propuneri inițiale. O imagine de intrare I este transformată într-o mulțime de adnotări:

$$A = \{a_1, a_2, \dots, a_n\}.$$

Fiecare adnotare conține un tip geometric, o etichetă semantică, o culoare de afișare și coordonate exprimate în sistemul imaginii originale. Tipurile geometrice folosite sunt: dreptunghi de încadrare, poligon și mască.

Fluxul este de tip *human-in-the-loop*: modelul propune, utilizatorul verifică și corectează. Această alegere este importantă deoarece segmentarea automată poate produce contururi incomplete, etichete greșite sau obiecte false pozitive. În loc să trateze predicția ca rezultat definitiv, aplicația o transformă într-o adnotare editabilă. Astfel, viteza modelelor de inteligență artificială este combinată cu judecata adnotatorului.

4.2. Algoritmi utilizați

La nivel funcțional, aplicația folosește trei categorii de algoritmi: algoritmi de generare a adnotărilor vizuale, algoritmi de asistență conversațională și algoritmi de sincronizare colaborativă. Această secțiune descrie rolul lor la nivel înalt, fără a intra în detalii de implementare.

4.2.1. Algoritmul AutoSeg

AutoSeg este algoritmul folosit pentru detecție și segmentare automată la nivelul întregii imagini. Intrările sale sunt imaginea curentă, lista de clase urmărite și un prag minim de încredere. Ieșirea este o listă de obiecte candidate, fiecare obiect putând conține o etichetă, un scor, o cutie de încadrare și un contur de segmentare.

Pașii logici ai algoritmului sunt:

1. utilizatorul alege clasele sau etichetele de interes;
2. imaginea este analizată de modelul extern;
3. predicțiile cu scor sub pragul ales sunt eliminate;
4. pentru fiecare predicție validă se extrage dreptunghiul de încadrare și, dacă există, masca obiectului;
5. rezultatul este convertit într-o adnotare editabilă.

O predicție AutoSeg poate fi reprezentată abstract prin:

$$d_i = (label_i, score_i, bbox_i, mask_i).$$

Componenta $bbox_i$ localizează rapid obiectul, iar $mask_i$ descrie conturul său. Dacă masca nu este disponibilă, dreptunghiul de încadrare rămâne o reprezentare minimă utilă. Avantajul algoritmului este că poate genera mai multe adnotări într-o singură rulare. Limita-

rea sa este dependența de modelul vizual și de compatibilitatea dintre etichetele cerute și obiectele pe care modelul le poate recunoaște.

4.2.2. Algoritmul AutoMask

AutoMask este algoritmul folosit pentru segmentare interactivă. Spre deosebire de AutoSeg, care caută obiecte în întreaga imagine, AutoMask pornește de la un indiciu spațial oferit de utilizator. Cel mai simplu indiciu este un punct $p = (x, y)$ ales în interiorul regiunii de interes. Modelul returnează o mască M asociată acelei regiuni:

$$f(I, p) \rightarrow M.$$

În modul de segmentare completă, algoritmul poate produce mai multe măști candidate:

$$f(I) \rightarrow \{M_1, M_2, \dots, M_m\}.$$

Acest mod este util când imaginea conține mai multe obiecte sau regiuni care pot fi pre-adnotate în bloc.

După generarea măștii, aplicația normalizează rezultatul pentru afișare și editare. Normalizarea urmărește două obiective: păstrarea formei relevante a obiectului și reducerea costului de randare. Această etapă este necesară deoarece o mască brută poate conține un număr foarte mare de puncte, iar manipularea ei directă poate încetini interfața.

4.2.3. Algoritmul de asistență LLM

Asistentul local folosește modele lingvistice mari pentru dialog cu utilizatorul. Rolul său nu este să modifice automat adnotările, ci să explice pași, să ajute la depanare și să ofere recomandări în fluxul de lucru. Conversația este reprezentată ca o listă de mesaje cu roluri: sistem, utilizator și asistent.

Un model lingvistic autoregresiv estimează următorul token pe baza contextului deja generat:

$$P(t_n \mid t_1, t_2, \dots, t_{n-1}).$$

Răspunsul este produs iterativ, token cu token. Parametrii precum dimensiunea contextului, temperatura, *top-p* și numărul maxim de tokeni controlează lungimea, variația și predictibilitatea răspunsului. În aplicație sunt folosite două moduri locale de rulare: `llama.cpp`, prin `llama-cpp-python`, și Ollama, prin client local.

4.2.4. Algoritmul de sincronizare colaborativă

Pentru colaborare, aplicația tratează modificările ca evenimente. Crearea, actualizarea și ștergerea unei adnotări sunt reprezentate prin mesaje care modifică starea sesiunii. Această stare poate fi descrisă prin relația:

$$S_{v+1} = \text{apply}(S_v, e),$$

unde S_v este starea sesiunii la versiunea v , iar e este evenimentul primit. Versiunea incrementală ajută la ignorarea evenimentelor vechi, iar identificatorul unic al evenimentului ajută la eliminarea duplicatelor.

Protocolul STOMP peste WebSocket este potrivit pentru acest flux deoarece permite modelul publicare/abonare: un client trimite un eveniment, iar ceilalți clienți abonați la aceeași sesiune îl primesc aproape în timp real. Prin această abordare, adnotările ma-

nuale și cele generate de AutoSeg sau AutoMask sunt sincronizate uniform.

4.3. Modele de inteligență artificială folosite

Algoritmii descriși anterior sunt susținuți de trei familii de modele: YOLO pentru detecție și segmentare, Mask2Former pentru segmentare externă și modele lingvistice bazate pe Transformer pentru asistența locală.

4.3.1. YOLO pentru AutoSeg

YOLO tratează detecția obiectelor ca o problemă de predicție directă. În loc să genereze mai întâi regiuni candidate și apoi să le clasifice separat, modelul procesează imaginea într-o singură trecere și prezice simultan poziția obiectelor, clasa lor și scorul de încredere. Imaginea este transformată într-o hartă de caracteristici, iar pe baza acesteia modelul estimează pentru fiecare obiect coordonatele cutiei de încadrare și probabilitatea apartenenței la o clasă. În variantele cu segmentare, aceeași predicție este completată cu informații despre masca obiectului. Pentru o aplicație de adnotare, această abordare este utilă deoarece reduce timpul dintre acțiunea utilizatorului și apariția propunerilor pe canvas.

În AutoSeg, modelul YOLO este folosit ca mecanism de pre-adnotare. Utilizatorul indică etichetele urmărite, iar modelul analizează întreaga imagine și întoarce o listă de obiecte candidate. Fiecare rezultat poate fi transformat fie într-un dreptunghi de încadrare, fie într-un poligon sau mască, în funcție de informațiile disponibile. Astfel, modelul nu este tratat ca un modul care decide definitiv adnotarea, ci ca un generator de propuneri care accelerează munca utilizatorului.

YOLOE extinde principiul YOLO către detecție și segmentare *open-vocabulary*, adică permite lucrul cu clase descrise prin prompturi textuale sau vizuale, nu doar cu o listă fixă de clase [11]. Arhitectura sa combină un extractor de caracteristici, o structură de agregare multi-scală de tip PAN și capete de predicție pentru regresie, segmentare și embedding. Prin alinierea dintre reprezentările vizuale și cele textuale, modelul poate răspunde la cereri mai flexibile decât un detector antrenat strict pe un set închis de categorii.

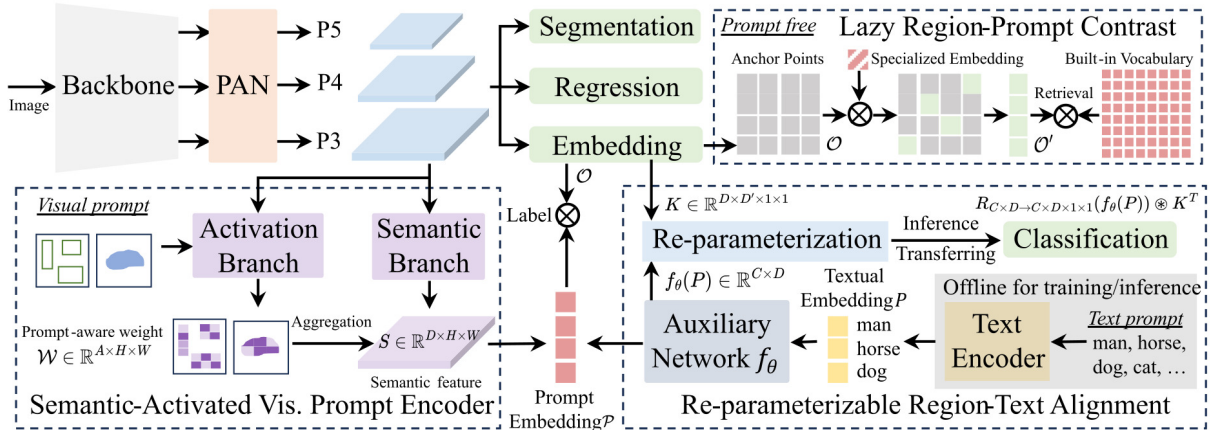


Figura 4.1: Arhitectura YOLOE folosită ca reper pentru AutoSeg

În figura 4.1 se observă separarea dintre extragerea de caracteristici, agregarea multi-scală și capetele de predicție. Pentru aplicația de adnotare, elementul relevant nu

este antrenarea modelului, ci forma rezultatului: o listă de obiecte candidate care pot deveni adnotări editabile. YOLOE ajută proiectul prin caracterul său generalist: același tip de model poate fi folosit pentru mai multe clase și contexte vizuale, fără ca interfața de adnotare să fie proiectată pentru un singur domeniu. Această flexibilitate susține scopul aplicației, deoarece utilizatorul poate lucra cu imagini variate și poate folosi AutoSeg ca punct de pornire rapid, urmat de corecție manuală.

4.3.2. Mask2Former pentru AutoMask

Mask2Former este folosit ca fundament pentru AutoMask deoarece problema principală aici nu este doar localizarea obiectului, ci obținerea unei regiuni precise. Modelul formulează segmentarea ca predicție de măști și poate aborda segmentarea semantică, segmentarea de instanțe și segmentarea panoptică folosind aceeași arhitectură generală [8]. Această proprietate este importantă pentru aplicație: indiferent dacă utilizatorul urmărește o regiune semantică sau un obiect individual, rezultatul util este tot o mască editabilă.

Arhitectura poate fi înțeleasă în trei etape. Prima etapă este extragerea caracteristicilor vizuale printr-un *backbone*. Acesta transformă imaginea într-o reprezentare internă care conține informații despre texturi, muchii, forme și obiecte. A doua etapă este *pixel decoder*-ul, care reconstruiește informația spațială necesară pentru segmentare la rezoluții utile. A treia etapă este decodorul Transformer, care lucrează cu interogări de obiect și produce predicții de clasă și măști.

Elementul central al Mask2Former este mecanismul de *masked attention*. În loc ca fiecare interogare să acorde atenție întregii imagini, atenția este restrânsă la regiuni candidate. Această restrângere reduce influența fundalului și permite rafinarea progresivă a conturului. Pentru AutoMask, acest lucru este util deoarece un punct selectat de utilizator poate fi transformat într-o regiune coerentă, iar regiunea rezultată poate fi corectată ulterior prin instrumentele de editare.

În figura 4.2, masca nu este doar un rezultat final, ci și un element folosit în atenția modelului. Această proprietate explică de ce Mask2Former este potrivit pentru generarea unor contururi inițiale care pot fi apoi corectate de utilizator. În proiect, AutoMask valorifică exact acest comportament: modelul extern produce o mască, iar aplicația o transformă într-o adnotare manipulabilă în canvas.

4.3.3. Modele lingvistice bazate pe Transformer

Modelele LLM folosite de aplicație sunt bazate pe arhitectura Transformer, introdusă ca mecanism de procesare secvențială prin atenție [28]. În modelele conversaționale moderne, varianta de tip *decoder-only* este folosită frecvent pentru generare autoregresivă. Modelul primește istoricul conversației, calculează relații de atenție între tokeni și produce următorul token.

În figura 4.3, blocurile principale sunt embedding-ul pozițional, atenția mascată, normalizarea și rețeaua feed-forward. Atenția mascată împiedică modelul să folosească tokeni viitori în timpul generării, menținând caracterul autoregresiv.

În proiect există două variante de rulare locală. `llama.cpp` permite încărcarea directă a unui model local și controlul parametrilor de inferență [29]. Ollama oferă un serviciu local care gestionează modelele disponibile și expune o interfață simplă pentru conversație [30]. Ambele variante susțin aceeași decizie arhitecturală: asistența conversațională rulează local, fără a trimite datele aplicației către un serviciu cloud.

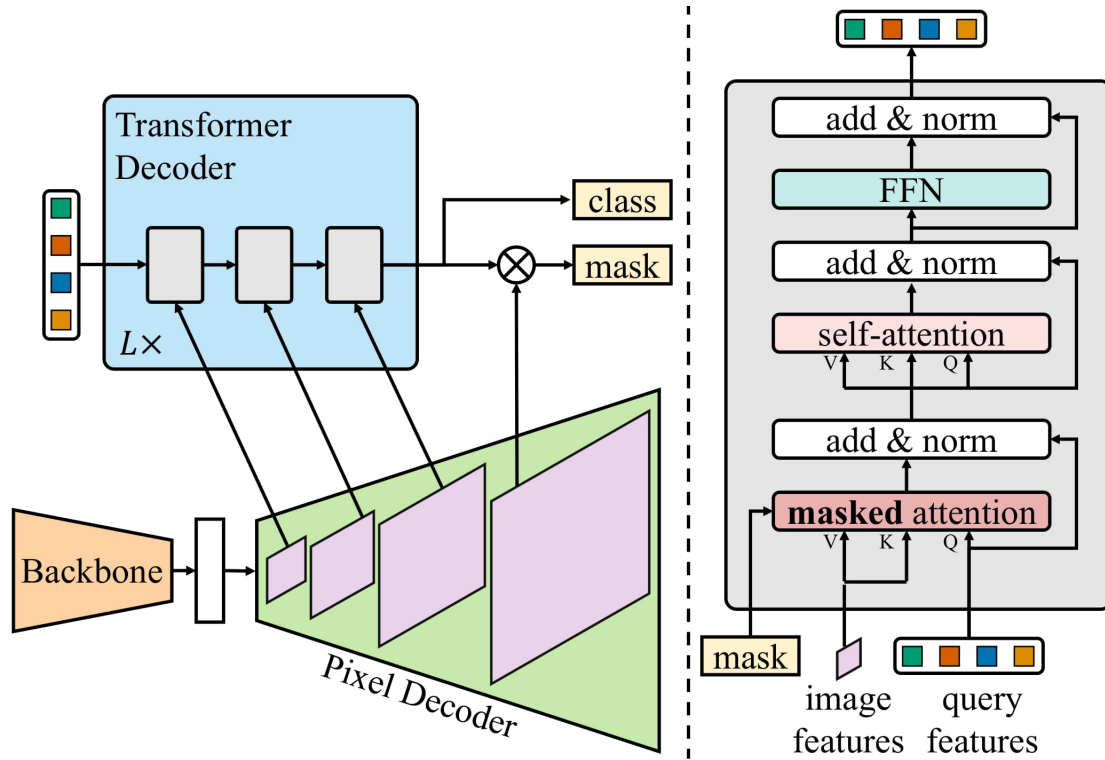


Figura 4.2: Arhitectura Mask2Former pentru predicția măștilor

4.4. Protocoale utilizate

Aplicația folosește protocoale diferite pentru tipuri diferite de comunicare. Operațiile punctuale, precum autentificarea sau obținerea stării unei sesiuni, sunt tratate prin HTTP/REST. Comunicarea continuă, precum chatul și sincronizarea adnotărilor, folosește WebSocket peste care este aplicat STOMP. Evenimentele interne dintre servicii sunt transmise prin RabbitMQ, folosind AMQP. Astfel, fiecare protocol este folosit acolo unde se potrivește natural: cerere-răspuns, timp real sau mesagerie asincronă.

4.4.1. HTTP/REST

HTTP/REST este folosit pentru acțiuni discrete, unde clientul trimite o cerere și primește un răspuns complet. În proiect, acest protocol acoperă autentificarea, înregistrarea utilizatorilor, validarea tokenului JWT, listarea sesiunilor, crearea sesiunilor și obținerea snapshotului unei camere colaborative. Este folosit și pentru transferul imaginilor temporare, deoarece fișierele imagine sunt mai potrivite pentru cereri HTTP decât pentru mesaje WebSocket.

JWT înseamnă *JSON Web Token* și reprezintă un token semnat care transportă identitatea utilizatorului între client și server. În proiect, el este emis după autentificare și este trimis ulterior la cererile protejate, astfel încât utilizatorul să nu retrimită parola la fiecare operație. Structura sa este împărțită conceptual în trei părți: header, payload și semnătură. Headerul descrie tipul tokenului și algoritmul folosit, payload-ul conține informațiile despre utilizator, iar semnătura permite verificarea integrității tokenului. Această structură este ilustrată în figura 4.4.

La nivel conceptual, REST reprezintă stratul de inițializare și recuperare. El oferă clientului starea de pornire, iar după aceea sincronizarea continuă poate fi făcută prin

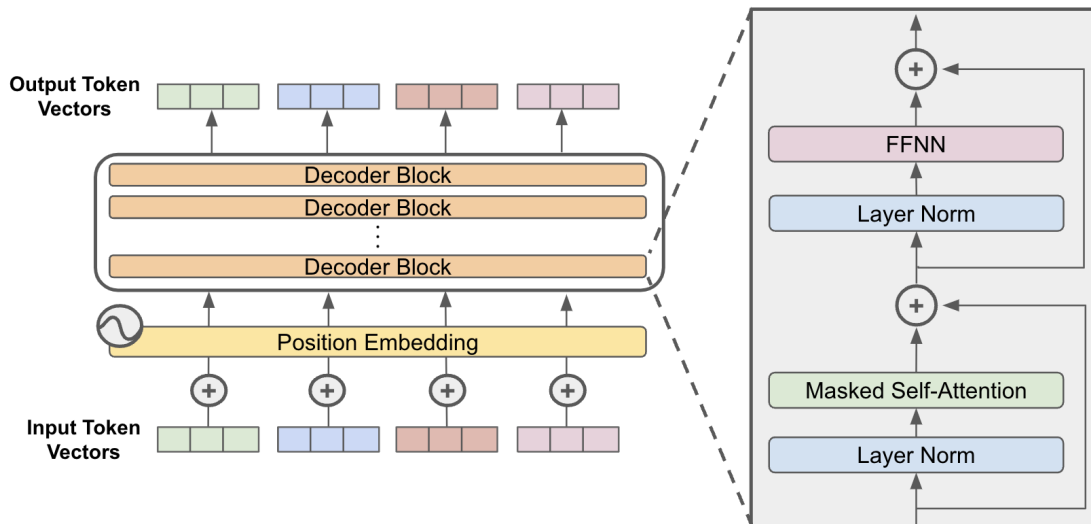


Figura 4.3: Structură generală de model Transformer de tip decoder-only

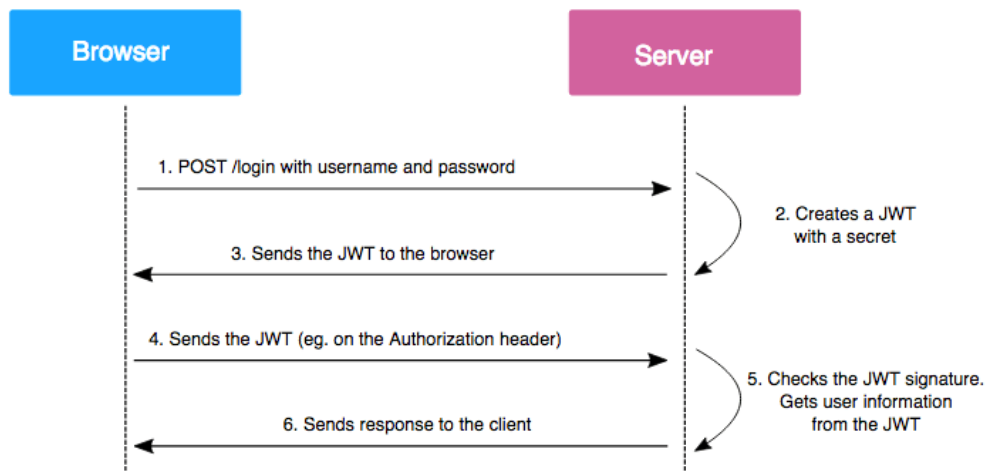


Figura 4.4: Structura generală a unui token JWT

canalul de timp real. Această separare este importantă deoarece nu toate operațiile aplicației au nevoie de comunicare permanentă. Autentificarea, încărcarea unei imagini sau cererea unui snapshot sunt operații finite, iar HTTP oferă un model clar pentru confirmarea succesului sau semnalarea erorilor.

În plus, REST creează un punct stabil de integrare între clientul desktop și serviciile backend. Clientul nu trebuie să cunoască starea internă a serverului, ci doar resursele pe care le poate cere sau modifica. Această abordare face ca pornirea unei sesiuni colaborative să fie predictibilă: mai întâi se obține starea inițială prin HTTP, apoi modificările live sunt primite prin WebSocket.

4.4.2. WebSocket și STOMP

WebSocket este folosit pentru comunicare persistentă între client și server. Acest lucru este necesar pentru funcțiile care trebuie actualizate imediat: chat, prezență și sincronizarea adnotărilor.

STOMP înseamnă *Simple Text Oriented Messaging Protocol* și definește un format

simplic de mesaje trimise peste o conexiune, inclusiv operații de tip trimitere și abonare. În proiect, este folosit peste WebSocket pentru a organiza mesajele pe destinații, astfel încât serverul să poată publica evenimente către topicuri sau cozi. Specificația protocolului este disponibilă la [22].

În aplicație, acest model permite separarea între mesaje globale și mesaje private. Evenimentele de colaborare sunt trimise către camera de lucru, iar mesajele de chat sunt livrate utilizatorilor implicați în conversație. Pentru adnotări mari, cum sunt măștile, sistemul poate fragmenta payload-ul și îl poate reasambla pe server, păstrând același model de eveniment. Avantajul principal al WebSocket este latența redusă. Într-o aplicație de adnotare colaborativă, un utilizator trebuie să vadă rapid modificările făcute de alt participant, altfel apar conflicte sau decizii luate pe o stare veche. Canalul persistent permite transmiterea evenimentelor imediat ce apar, fără interogări repetate către server.

4.4.3. SockJS ca fallback

SockJS este disponibil ca fallback pentru endpointurile WebSocket. Rolul său este să permită conectarea și în medii unde WebSocket direct nu este disponibil sau este restricționat. Pentru clientul desktop scenariul principal rămâne WebSocket nativ, dar suportul SockJS face backend-ul mai ușor de reutilizat de alte interfețe viitoare. În proiect, SockJS este configurat în serviciile de chat și adnotare, la același endpoint WebSocket, ca alternativă de transport. Fiind plasat la nivelul transportului, nu schimbă modelul de colaborare și nici formatul evenimentelor. Această alegere este utilă deoarece serviciile backend nu rămân dependente de un singur tip de client.

4.4.4. AMQP și RabbitMQ

AMQP, prin RabbitMQ, este folosit pentru evenimente asincrone între servicii. În proiect, autentificarea poate publica evenimente interne, iar celelalte servicii le pot consuma fără apeluri directe între module. Acest model reduce cuplarea dintre servicii: un serviciu anunță că s-a întâmplat un eveniment, iar consumatorii interesați reacționează.

RabbitMQ este rulat ca serviciu separat în infrastructura Docker și devine un canal comun pentru comunicarea internă. Prin această separare, backend-ul poate fi extins mai ușor cu alte servicii care ascultă sau publică evenimente. Pentru aplicație, acest lucru este relevant mai ales în relația dintre autentificare și celelalte servicii. După crearea unui utilizator, informația poate fi propagată asincron fără ca serviciul de autentificare să depindă direct de implementarea chatului sau a modului de adnotare. Astfel, serviciile rămân mai independente, iar schimbările dintr-un modul au impact mai mic asupra celorlalte.

AMQP completează REST și WebSocket printr-un rol diferit. REST este folosit pentru cereri inițiate de client, WebSocket pentru evenimente live către client, iar AMQP pentru evenimente interne între servicii. Împreună, aceste protocoale formează un model de comunicare potrivit pentru o aplicație distribuită cu client desktop, servicii backend și infrastructură de mesagerie.

4.5. Frameworkuri și platforme

Sistemul este împărțit între un client desktop și mai multe servicii backend, iar tehnologiile folosite reflectă această separare. Clientul are nevoie de interacțiune grafică, canvas, evenimente locale și procese asincrone, în timp ce backend-ul are nevoie de endpointuri HTTP, WebSocket, securitate, mesagerie și rulare izolată în containere.

4.5.1. Qt și PyQt6 pentru clientul desktop

Clientul desktop este scris în Python folosind PyQt6, care oferă legătura dintre interfața Qt și logica aplicației. Fereastra principală orchestrează panoul de navigare, canvasul de adnotare, panoul de proprietăți și bara de unelte. Canvasul este zona centrală a aplicației și permite încărcarea imaginii, zoom, pan, desenarea de dreptunghiuri, poligoane și măști, selecția obiectelor și actualizarea vizuală a adnotărilor.

Python are rol de orchestrare locală: leagă interfața grafică, fișierele locale, serializarea adnotărilor, apelurile REST, conexiunile WebSocket și procesele externe. AutoSeg și AutoMask pot fi executabile sau scripturi separate, iar clientul le transmite argumentele necesare și interpretează răspunsul JSON ca adnotări editabile. Aceeași structură este folosită și pentru LLM-uri: Ollama și `llama.cpp` sunt apelate local, iar răspunsurile lor sunt livrate în interfață prin worker-e.

PyQt6 permite separarea operațiilor lente în worker-e care emit semnale la finalizare, la eroare sau la primirea unui fragment de răspuns. Modelele AutoSeg, AutoMask, autentificarea, conexiunile STOMP și LLM-urile nu rulează direct în threadul interfeței. Această decizie păstrează aplicația responsivă chiar și când un model extern sau o cerere de rețea durează mai mult.

4.5.2. PyTorch pentru modelele externe

PyTorch este relevant în proiect prin modelele externe de viziune. AutoSeg și AutoMask pot rula scripturi sau executabile construite în jurul unor modele de detecție și segmentare. Aceste modele folosesc frecvent PyTorch pentru încărcarea greutăților, inferență pe CPU sau GPU și prelucrarea tensorilor. Aplicația desktop nu depinde direct de structura internă a modelului, dar trebuie să îi poată transmite corect imaginea și să interpreteze rezultatul.

Această separare între UI și model are două avantaje. Primul este flexibilitatea: se poate schimba modelul fără modificarea canvasului sau a formatului intern de adnotare, atâta timp cât ieșirea rămâne compatibilă. Al doilea este izolarea erorilor: dacă modelul extern eșuează, aplicația poate afișa o eroare și poate continua să funcționeze manual. Pentru o aplicație de adnotare, această izolare este importantă deoarece utilizatorul nu trebuie blocat de o singură componentă AI.

4.5.3. Java și Spring Boot pentru serviciile backend

Backend-ul este împărțit în servicii Spring Boot: autentificare, chat și adnotare colaborativă. Această împărțire reflectă trei responsabilități distincte. Serviciul de autentificare gestionează conturile, parolele, JWT-ul și validarea accesului. Serviciul de chat gestionează mesajele private, istoricul conversațiilor și prezența utilizatorilor. Serviciul de adnotare gestionează sesiunile colaborative, imaginile temporare și evenimentele de adnotare.

Spring Boot este folosit pentru expunerea endpointurilor REST, configurarea WebSocket/STOMP, integrarea cu RabbitMQ și aplicarea regulilor de securitate. În serviciul de adnotare, controllerul REST gestionează lista de sesiuni și snapshoturile, iar controllerul WebSocket gestionează evenimentele live. În serviciul de chat, controllerul STOMP primește cereri de trimitere mesaj, istoric și prezență. Această organizare separă clar cererile punctuale de fluxurile în timp real.

Un aspect important este folosirea obiectelor DTO pentru mesajele dintre client și server. În loc ca fiecare serviciu să accepte structuri arbitrare, datele sunt împachetate în

obiecte cu câmpuri explicite: cereri de creare sesiune, răspunsuri de sesiune, envelope-uri WebSocket, mesaje chat și metadata de imagine. Acest model reduce ambiguitatea și permite validarea datelor la intrarea în serviciu.

4.5.4. Docker și orchestrarea serviciilor

Docker este folosit pentru rularea coordonată a serviciilor backend și a infrastructurii. Configurația include un reverse proxy, RabbitMQ, Postgres, serviciul de autentificare, serviciul de chat și serviciul de adnotare. Toate aceste componente sunt conectate într-o rețea comună, iar dependențele sunt declarate explicit.

Reverse proxy-ul oferă un punct de intrare unic pentru client. În loc ca aplicația desktop să cunoască porturile interne ale fiecărui serviciu, proxy-ul poate direcționa cererile către serviciul potrivit pe baza rutei. De exemplu, rutele de adnotare sunt expuse sub prefixul public `/annotation`, în timp ce serviciile interne rulează pe porturi separate.

Postgres este folosit pentru persistența conturilor din serviciul de autentificare. RabbitMQ este folosit pentru evenimentele dintre servicii. Serviciul de chat folosește implicit o bază H2 în memorie pentru reprezentarea locală a utilizatorilor și păstrează conversațiile active într-o structură internă. Serviciul de adnotare păstrează colaborarea în memorie, inclusiv sesiunile, utilizatorii online, adnotările și imaginile temporare. Această alegere este adecvată pentru un MVP, unde scopul este validarea fluxului colaborativ înaintea introducerii unei persistențe complete pentru toate datele.

4.6. Modele abstracte

Modelele abstracte descriu conceptele stabile ale sistemului, independent de modulele concrete care le implementează. Ele nu descriu fluxul aplicației, ci entitățile de bază și relațiile dintre ele. În proiect sunt relevante patru modele: acces, colaborare, adnotare și stocare.

4.6.1. Modelul de acces

Modelul de acces are ca entitate principală utilizatorul. Utilizatorul este identificat prin email, iar după autentificare este reprezentat în sistem printr-un token JWT. Tokenul separă identitatea utilizatorului de credențialele sale, astfel încât accesul la servicii să poată fi verificat fără retrimiteria parolei.

Autorizarea este exprimată prin relația dintre utilizator și sesiune. Un utilizator conectat la o sesiune poate acționa asupra resurselor acelei sesiuni în funcție de rolul său. În versiunea actuală, rolul principal este cel de editor, dar modelul permite extinderea cu roluri mai restrictive.

4.6.2. Modelul de colaborare

Modelul de colaborare are ca entitate centrală sesiunea. O sesiune grupează utilizatori, o imagine de lucru și o colecție de adnotări. Starea ei se modifică prin evenimente, iar fiecare eveniment reprezintă o schimbare conceptuală: intrare în sesiune, ieșire, imagine disponibilă sau modificare de adnotare.

Pentru a menține coerența, sesiunea are o versiune. Versiunea nu descrie implementarea, ci ordinea logică a schimbărilor. Ea permite diferențierea între starea curentă și evenimente vechi sau duplicate.

4.6.3. Modelul de adnotare

Modelul de adnotare descrie obiectele vizuale care apar peste imagine. O adnotare are un identificator, o etichetă, un tip geometric și coordonate. Tipurile principale sunt dreptunghiul, poligonul și masca. Fiecare tip exprimă un nivel diferit de precizie: localizare aproximativă, contur vectorial sau regiune densă.

Același model se aplică atât adnotărilor manuale, cât și celor generate de AutoSeg sau AutoMask. Această uniformizare este importantă deoarece sistemul nu trebuie să trateze separat originea adnotării. După creare, toate adnotările sunt obiecte editabile ale aceluiași model abstract.

4.6.4. Modelul de stocare

Modelul de stocare separă datele după durata lor de viață. Datele de identitate trebuie să fie persistente, deoarece sunt necesare între sesiuni de lucru diferite. Datele de colaborare pot fi temporare, deoarece descriu o activitate live asupra unei imagini curente. Datele finale ale datasetului sunt produse prin salvare și export.

Această separare clarifică responsabilitățile sistemului. Persistența identității ține de autentificare, starea colaborativă ține de sesiunea activă, iar artefactele finale țin de clientul care exportă adnotările. Astfel, modelul de stocare nu este unul unic, ci adaptat tipului de date gestionat.

4.7. Structura logică și funcțională a aplicației

Structura aplicației poate fi privită pe trei niveluri. Primul nivel este clientul desktop, unde utilizatorul încarcă imagini, creează adnotări și pornește modelele automate. Al doilea nivel este format din serviciile backend, care gestionează autentificarea, chatul și colaborarea. Al treilea nivel este infrastructura, formată din Postgres, RabbitMQ, reverse proxy și containere.

4.7.1. Clientul desktop

Clientul desktop este nivelul în care utilizatorul execută efectiv fluxul de adnotare. El pornește de la selectarea imaginii, continuă cu adăugarea sau corectarea adnotărilor și se încheie cu salvarea sau exportul datasetului. Din perspectiva structurii logice, clientul nu este doar o interfață grafică, ci coordonatorul operațiilor locale: gestionează imaginea curentă, starea adnotărilor, modificările nesalvate și legătura cu serviciile externe.

Funcționalitatea manuală rămâne baza aplicației, iar AutoSeg și AutoMask sunt integrate ca acceleratori ai aceluiași flux. Rezultatele generate automat nu sunt tratate separat, ci devin obiecte editabile în aceeași stare locală. Această alegere permite utilizatorului să alterneze natural între desenare manuală, propuneri AI și corecții.

În modul colaborativ, clientul are și rol de mediator între starea locală și starea comună a sesiunii. La intrarea într-o cameră, clientul preia un snapshot, iar ulterior aplică evenimente incrementale primite de la server. Când modificarea este făcută local, ea este transformată într-un eveniment trimis către backend; când modificarea vine de la alt utilizator, ea este aplicată fără retransmitere. Astfel se evită buclele de sincronizare și se păstrează coerența dintre participanți.

4.7.2. Worker-ele locale

Worker-ele sunt componente locale care execută operații potențial lente. Ele separă logica de fundal de interfața grafică. În proiect există worker-e pentru autentificare, chat STOMP, colaborare STOMP, AutoSeg, AutoMask, Ollama și `llama.cpp`.

Worker-ele AutoSeg și AutoMask pornesc procese externe. Ele construiesc lista de argumente, pornesc executabilul sau scriptul configurat și așteaptă rezultatul. Execuția este făcută fără shell, cu argumente explicite, pentru a evita interpretarea necontrolată a comenzii. Rezultatul este citit din stdout și interpretat ca JSON. Dacă procesul depășește timpul permis, el poate fi oprit.

Worker-ele LLM sunt diferite deoarece produc răspunsuri incrementale. În loc să aștepte întregul răspuns, ele emit tokeni pe măsură ce apar. Fereastra de chatbot poate afișa textul gradual, ceea ce face interacțiunea mai naturală. Această tehnică este utilă mai ales pentru modele locale, unde timpul de generare poate varia în funcție de hardware.

Worker-ele de comunicație mențin conexiunile cu backend-ul. Pentru chat și colaborare, ele deschid conexiuni WebSocket, trimit frame-uri STOMP și ascultă mesajele primite. Ele decuplează rețeaua de interfața grafică. Astfel, un timeout, o reconectare sau un mesaj invalid nu blochează desenarea pe canvas.

4.7.3. Serviciile backend

Serviciul de autentificare este responsabil pentru identitate. El primește cereri de login și register, validează datele, generează JWT și publică evenimente de înregistrare. Acest serviciu este singurul care are persistență Postgres pentru conturi. Prin această separare, datele sensibile de autentificare nu sunt distribuite în celelalte module.

Serviciul de chat este responsabil pentru comunicarea directă între utilizatori. El primește mesaje prin STOMP, verifică utilizatorii și livrează mesajele prin cozi private. De asemenea, poate trimite istoricul unei conversații și snapshoturi de prezență. Pentru scenariul curent, istoricul este păstrat în memorie, ceea ce simplifică funcționarea și reduce nevoia unei scheme persistente complexe.

Serviciul de adnotare este responsabil pentru colaborarea pe imagini. El expune endpointuri REST pentru sesiuni și imagini temporare și endpointuri WebSocket pentru evenimente live. Starea unei sesiuni include utilizatorii, adnotările și versiunea. Serverul validează membership-ul înainte de aplicarea evenimentelor, deduplicatează evenimentele prin identificator și transmite rezultatul către topicul sesiunii.

Împărțirea în servicii permite dezvoltarea independentă a responsabilităților. Autentificarea poate evolua separat de chat. Chatul poate fi extins fără să schimbe formatul adnotărilor. Serviciul de adnotare poate primi o bază de date persistentă în viitor fără să schimbe complet clientul desktop.

4.7.4. Persistență și export

Persistența aplicației nu se reduce la o singură bază de date. Datele de autentificare sunt persistente în Postgres. Datele de chat și colaborare sunt predominant volatile. Datele finale de adnotare sunt salvate și exportate de client. Această împărțire reflectă prioritățile proiectului.

Pentru dataseturi, forma finală trebuie să fie ușor de folosit în antrenare. De aceea, aplicația oferă export JSON intern, COCO și YOLO. Formatul intern păstrează informațiile necesare pentru reîncărcarea adnotărilor în aplicație. COCO și YOLO sunt

formate de schimb, utile pentru fluxuri de antrenare. Astfel, clientul desktop este și instrument de lucru, și instrument de pregătire a datelor.

În cazul măștilor, stocarea trebuie să evite fișiere excesiv de mari. Reprezentarea brută a unei măști poate conține foarte multe puncte. De aceea, aplicația folosește reprezentări compacte și normalizare pentru a păstra datele manipulabile. Acest aspect are impact direct asupra performanței la salvare, încărcare, randare și sincronizare.

4.7.5. Flux funcțional complet

Un flux complet începe cu autentificarea utilizatorului. Clientul trimite emailul și parola către serviciul de autentificare și primește un JWT. Tokenul este păstrat local și folosit pentru cererile REST și pentru conectarea WebSocket. După autentificare, utilizatorul poate lucra local sau poate intra într-o sesiune colaborativă.

În modul local, utilizatorul deschide un folder de imagini, selectează o imagine și creează adnotări. Poate folosi manual dreptunghiuri, poligoane sau măști. Poate porni AutoSeg pentru a obține propuneri pe baza etichetelor. Poate folosi AutoMask pentru o regiune indicată prin click sau pentru segmentare completă. Rezultatele automate devin obiecte editabile și pot fi corectate exact ca adnotările manuale.

În modul colaborativ, utilizatorul creează sau se alătură unei sesiuni. Clientul cere snapshotul sesiunii și se abonează la topicurile STOMP relevante. Dacă utilizatorul încarcă o imagine, aceasta este trimisă prin HTTP, iar ceilalți clienți sunt anunțați prin WebSocket. Dacă utilizatorul creează o adnotare, aceasta este trimisă ca eveniment, serverul o aplică și apoi o publică tuturor participanților.

Chatul funcționează în paralel cu adnotarea. Utilizatorii pot comunica direct, pot cere istoricul conversației și pot vedea informații de prezență. Această funcționalitate susține colaborarea deoarece adnotatorii pot discuta despre etichete, corecții sau neclarități fără să părăsească aplicația.

La final, utilizatorul salvează sau exportă datele. Exportul poate fi folosit ca dataset pentru antrenare sau evaluare. În acest fel, aplicația acoperă întregul traseu: încărcare imagine, adnotare manuală, asistență AI, colaborare, comunicare și export.

4.8. Justificarea integrării modelelor

AutoSeg și AutoMask sunt complementare. AutoSeg este eficient pentru pre-adnotarea mai multor obiecte pe baza unei liste de clase, în timp ce AutoMask este mai potrivit pentru situațiile în care utilizatorul poate indica direct regiunea dorită. Prin combinarea lor, aplicația nu depinde de un singur mod de interacțiune cu modelul vizual.

Modelele LLM completează fluxul vizual prin suport conversațional. Ele nu produc automat date de antrenare și nu decid asupra calității adnotărilor, ci asistă utilizatorul în folosirea aplicației. Separarea dintre modelele vizuale, modelele lingvistice și mecanismul colaborativ reduce cuplarea sistemului: fiecare componentă poate fi schimbată fără a modifica modelul conceptual al adnotării.

În ansamblu, soluția folosește modelele AI ca instrumente de accelerare, nu ca înlocuitor al validării umane. Această abordare este potrivită pentru o aplicație de adnotare, unde scopul principal este producerea unor date coerente, verificabile și ușor de corectat.

Capitolul 5. Proiectare de detaliu și implementare

Capitolul de față descrie modul în care soluția propusă a fost proiectată și implementată, pornind de la arhitectura generală și coborând treptat spre componentele concrete ale aplicației. După prezentarea imaginii de ansamblu, capitolul detaliază clientul desktop, modelul intern de adnotare, mecanismele de salvare și export, integrarea worker-elor pentru modele AI, serviciile backend, comunicarea în timp real, persistența datelor și infrastructura de rulare. Scopul capitolului nu este reluarea elementelor teoretice din analiza anterioară, ci explicarea deciziilor de proiectare care leagă modulele implementate într-un sistem funcțional.

5.1. Arhitectura generală a sistemului

Sistemul este organizat în jurul unei aplicații desktop de adnotare, care comunică atât cu modele locale de inteligență artificială, cât și cu un backend împărțit în servicii specializate. La nivel conceptual, aplicația poate fi privită ca o combinație între o zonă interactivă, folosită direct de utilizator pentru încărcarea imaginilor și editarea adnotărilor, o zonă de procesare automată, responsabilă de operațiile AI, și o zonă de servicii, responsabilă de autentificare, chat, colaborare și sincronizare.

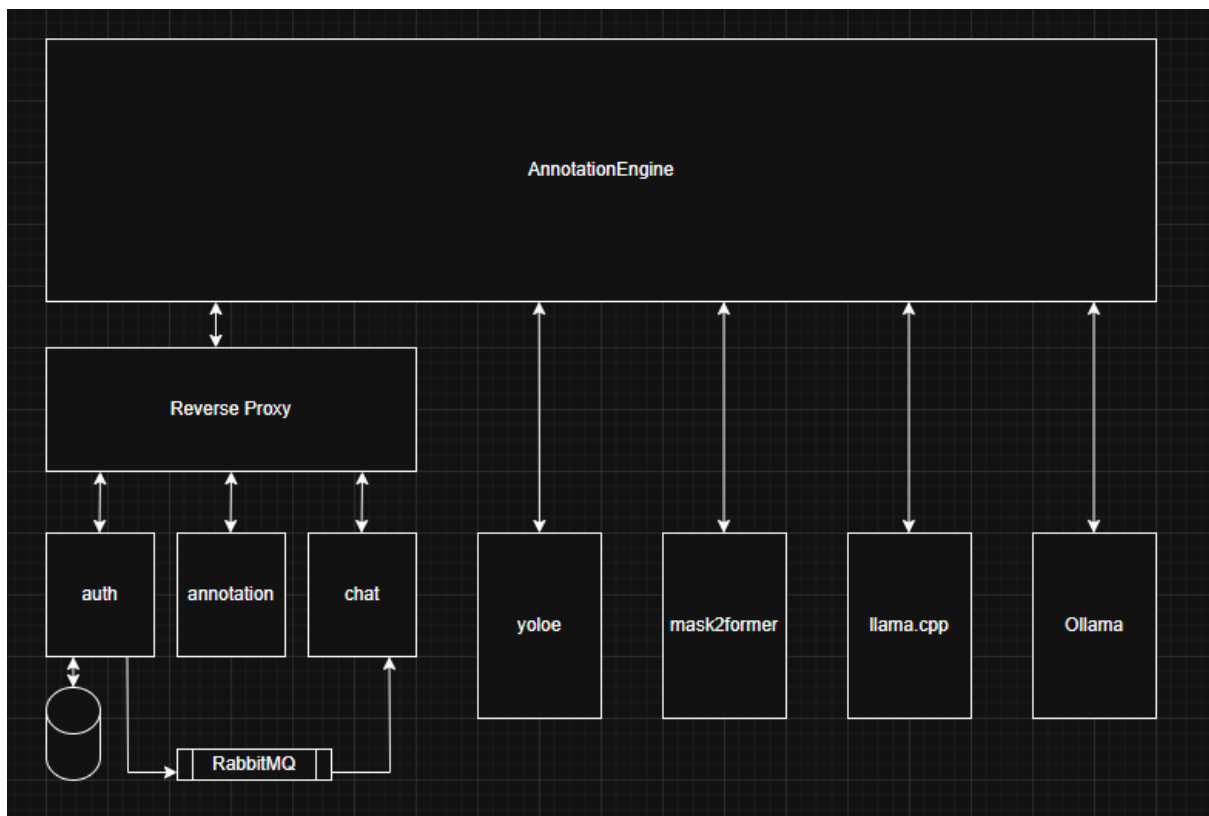


Figura 5.1: Arhitectura generală a aplicației

Figura 5.1 prezintă legătura dintre componentele principale ale proiectului. În partea de client se află modulul **AnnotationEngine**, aplicația desktop scrisă în Python cu PyQt6. Acesta concentrează interacțiunea utilizatorului cu imaginile, uneltele de desenare, lista de etichete, mecanismele de import/export și ferestrele auxiliare pentru autentificare, chat și chatbot local. Din perspectiva utilizatorului, clientul desktop este punctul central al sistemului, deoarece toate acțiunile pornesc din interfața grafică.

A doua zonă importantă este **ColaborativeServer**, care grupează serviciile backend dezvoltate cu Spring Boot. Backend-ul nu este implementat ca o singură aplicație monolitică, ci ca un set de servicii cu responsabilități separate: serviciul de autentificare gestionează utilizatorii și token-urile JWT, serviciul de chat gestionează conversațiile și prezența, iar serviciul de annotation gestionează sesiunile colaborative și starea comună a adnotărilor. Această separare face ca funcțiile aplicației să poată evolua independent și reduce dependența dintre autentificare, comunicare și sincronizarea efectivă a lucrului pe imagini.

A treia zonă este reprezentată de directorul **research**, folosit pentru experimentare, testare și comparație între modele sau variante de integrare. Această parte nu este interfața principală a utilizatorului final, dar are rol practic în proiect: aici pot fi testate modele precum YOLOE sau Mask2Former, pot fi verificate intrări și ieșiri, iar rezultatele stabile pot fi apoi integrate în fluxurile controlate de aplicația desktop. Separarea zonei de cercetare de codul clientului ajută la păstrarea aplicației mai curate și permite modificarea experimentelor fără a afecta direct interfața utilizatorului.

O decizie importantă de proiectare este separarea dintre client, modelele externe și backend. Clientul desktop trebuie să rămână responsabil, deoarece utilizatorul poate lucra cu imagini, măști și forme geometrice care necesită randare și interacțiune directă. Modelele AI pot avea timpi de execuție mai mari, pot depinde de biblioteci specializate și pot necesita procese separate. Din acest motiv, ele nu sunt apelate direct în firul principal al interfeței, ci prin worker-e sau procese externe. În acest mod, operații precum segmentarea automată, generarea de măști sau conversația cu un LLM local pot rula fără a bloca aplicația.

Pentru zona de adnotare automată, clientul folosește worker-e dedicate care pornesc sau comunică cu executabile și scripturi externe. AutoSeg este responsabil pentru detecție și segmentare automată pe baza unui model YOLO, în timp ce AutoMask folosește un model de segmentare pe mască pentru a produce contururi mai precise în jurul obiectelor. Rezultatele acestor procese sunt transformate în structuri compatibile cu modelul intern de adnotare al aplicației, astfel încât utilizatorul să le poată corecta, redenumi sau exporta la fel ca pe adnotările create manual.

Un principiu similar este folosit și pentru integrarea modelelor conversaționale locale. Aplicația nu depinde de un serviciu cloud pentru chatbot, ci poate folosi local **llama.cpp** prin **llama-cpp-python** sau Ollama prin clientul său local. În arhitectură, aceste modele sunt tratate ca servicii locale auxiliare: interfața trimite mesajele către worker, worker-ul gestionează apelul către motorul LLM, iar răspunsul este readus în aplicație sub formă de mesaj afișabil. Această abordare păstrează controlul asupra resurselor locale și face posibilă rularea funcțiilor AI chiar și în scenarii în care conectivitatea externă nu este o condiție de bază.

Backend-ul completează funcțiile locale prin mecanisme de colaborare și sincronizare. Autentificarea este expusă prin endpoint-uri HTTP/REST, iar sesiunile de lucru și

schimbul de evenimente sunt gestionate prin WebSocket și STOMP. În practică, clientul se autentifică, primește un token JWT, apoi îl folosește pentru a accesa funcțiile care necesită identitatea utilizatorului. Pentru operațiile în timp real, evenimentele nu sunt trimise ca cereri izolate, ci ca mesaje publicate către canale asociate sesiunilor, ceea ce permite mai multor clienți să vadă modificări și mesaje pe măsură ce acestea apar.

Infrastructura de rulare este definită cu Docker, astfel încât serviciile backend și dependențele lor să poată fi pornite într-un mod controlat. În configurația proiectului, Traefik are rol de reverse proxy și direcționează cererile către serviciile corespunzătoare. RabbitMQ este folosit ca broker AMQP pentru evenimente între servicii, de exemplu pentru propagarea unor informații despre utilizatori între modulul de autentificare și celelalte module. Pentru datele persistente ale autentificării se folosește Postgres, iar componentele de chat și colaborare păstrează o parte din stare în memorie, deoarece natura lor este mai apropiată de sesiuni active și evenimente de lucru.

Această arhitectură susține două moduri de lucru care apar frecvent în aplicație. În modul individual, utilizatorul poate încărca imagini, crea adnotări manuale, apela modele AI locale și exporta rezultatele fără a depinde de colaborarea cu alți utilizatori. În modul colaborativ, aceleași acțiuni sunt completate de autentificare, camere de lucru, mesaje și sincronizare prin backend. Din acest motiv, clientul desktop este proiectat ca un punct de coordonare: el poate lucra local cu fișiere și worker-e, dar poate folosi și serviciile backend atunci când este necesară partajarea stării între mai mulți utilizatori.

Prin această împărțire, proiectul rămâne modular. **AnnotationEngine** se concentrează pe experiența de adnotare și pe integrarea locală a modelelor, **ColaborativeServer** gestionează identitatea și comunicarea între utilizatori, iar zona **research** permite evoluția componentelor AI fără a destabiliza aplicația principală. În secțiunile următoare, fiecare dintre aceste zone este detaliată din perspectiva implementării: structura clientului, reprezentarea adnotărilor, worker-ele AI, fluxurile de comunicare și serviciile backend care susțin colaborarea.

5.2. Structura clientului desktop PyQt

Clientul desktop reprezintă partea aplicației cu care utilizatorul lucrează direct. În proiect, această componentă este implementată în directorul **AnnotationEngine** și are rolul de a transforma funcționalitățile sistemului într-un flux de lucru vizual: deschiderea unui set de imagini, desenarea adnotărilor, ajustarea proprietăților, apelarea modelelor externe, exportul datasetului și conectarea la funcțiile colaborative. Spre deosebire de backend, care este împărțit în servicii, clientul desktop este o aplicație interactivă, în care timpul de răspuns al interfeței și claritatea acțiunilor sunt importante pentru utilizator.

5.2.1. Rolul clientului desktop în sistem

Aplicația pornește din fișierul **app.py**. Acesta creează obiectul **QApplication**, setează numele aplicației, aplică stilul vizual de bază și deschide fereastra principală **MainWindow**. Tot aici există și o etapă de inițializare pentru backend-urile AI care pot fi livrate împreună cu aplicația. Funcția de inițializare caută în directoare precum **inference_backends**, **dist** sau **models** executabile pentru AutoMask, executabile YOLO și fișiere de ponderi **.pt**, apoi completează valorile lipsă în **QSettings**. Astfel, aplicația poate porni cu o configurare inițială utilă, fără ca utilizatorul să introducă manual toate

căile atunci când executabilele se află deja lângă program.

Din punct de vedere funcțional, **AnnotationEngine** poate fi folosit în două moduri. Primul este modul local, în care utilizatorul încarcă imagini dintr-un folder, creează adnotări manuale sau automate și exportă rezultatele în formate potrivite pentru antrenarea sau testarea unor modele. Al doilea este modul conectat, în care aplicația folosește autentificarea, chatul și colaborarea în timp real. Această dublă natură explică de ce în client există atât componente de interfață pur locale, precum canvasul și panourile de editare, cât și componente care comunică prin HTTP sau WebSocket cu serviciile din **ColaborativeServer**.

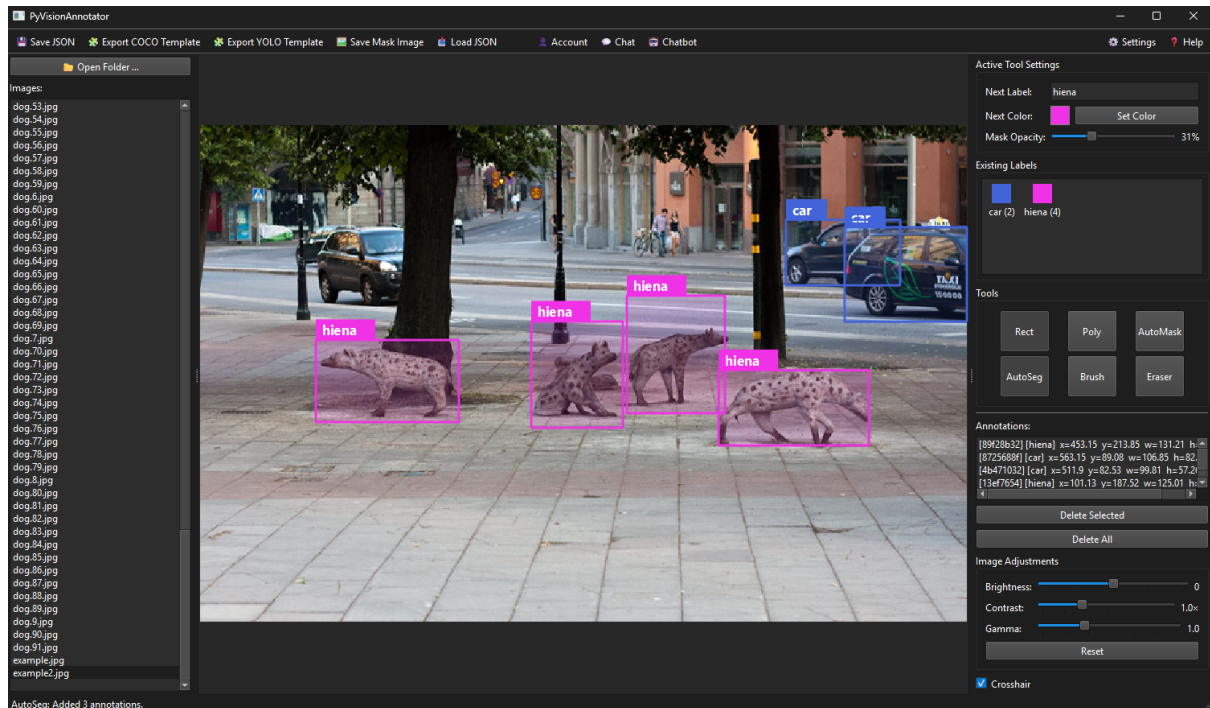


Figura 5.2: Interfața principală a aplicației PyVisionAnnotator

După cum se observă în Figura 5.2, interfața este împărțită în zone care urmăresc fluxul natural de lucru al utilizatorului. În partea stângă se află lista de imagini din folderul curent, zona centrală este canvasul pe care se afișează imaginea și adnotările, iar panoul din dreapta conține proprietățile obiectului selectat, unelte active și lista de adnotări. Această împărțire permite utilizatorului să navigheze rapid între imagini, să lucreze direct pe conținutul vizual și să modifice etichetele, culorile sau setările fără a părăsi fereastra principală.

Clientul desktop nu integrează modelele AI direct în codul UI. AutoSeg, AutoMask, Ollama și llama.cpp sunt tratate ca servicii locale sau procese externe, apelate prin worker-e. Această decizie păstrează interfața mai stabilă și reduce dependența dintre codul de desenare și bibliotecile de inferență. Dacă un model este schimbat, ambalat într-un executabil nou sau rulat cu alte argumente, fluxul general al aplicației rămâne același: UI-ul colectează parametrii, worker-ul execută operația, rezultatul este convertit în adnotări.

5.2.2. MainWindow ca orchestrator al aplicației

Clasa `MainWindow`, definită în `ui/main_window/main_window.py`, este centrul de coordonare al clientului desktop. Rolul ei nu este să implementeze toate detaliile fiecărei funcții, ci să construiască interfața, să conecteze semnalele dintre componente și să păstreze starea comună necesară pentru fluxurile care traversează mai multe module. Din acest motiv, fereastra principală conține referințe către managerul de adnotări, canvas, panouri, worker-ele active și ferestrele auxiliare pentru chat sau chatbot.

Construirea interfeței este concentrată în metoda `_build_ui()`. În această metodă se creează `AnnotationCanvas`, `LeftPanel`, `RightPanel` și `ToolbarPanel`. Cele trei zone principale ale ferestrei sunt așezate într-un `QSplitter` orizontal, cu panoul de imagini în stânga, canvasul în centru și panoul de proprietăți în dreapta. Canvasul primește factorul de întindere cel mai mare, deoarece el este zona de lucru efectivă, în timp ce panourile laterale sunt zone de control și navigare.

Metoda `_connect_signals()` definește fluxul reactiv al aplicației. Panourile nu modifică direct toate celelalte componente, ci emit semnale care sunt conectate la metode din `MainWindow`, `AnnotationCanvas` sau `AnnotationManager`. Această separare este importantă pentru întreținere: un buton din panoul drept nu are nevoie să cunoască detaliile de încărcare a imaginilor sau de colaborare, ci doar să emită o intenție, de exemplu schimbarea unelei active. Fereastra principală decide apoi unde ajunge acea intenție.

```
self._left.image_load_requested.connect(self._on_image_load_requested)
self._right.tool_changed.connect(self._canvas.set_tool)
self._canvas.image_loaded.connect(self._on_image_loaded)
self._manager.annotation_added.connect(self._on_data_changed)
```

Fragmentul de mai sus ilustrează stilul de comunicare folosit în aplicație. Panoul stâng cere încărcarea unei imagini, panoul drept schimbă unealta activă a canvasului, canvasul anunță că imaginea a fost încărcată, iar managerul anunță că datele s-au modificat. În locul unui flux liniar rigid, aplicația folosește semnale și sloturi pentru a păstra componentele decuplate. Acest model este potrivit pentru aplicația de adnotare, deoarece multe acțiuni pot porni din interfață: click în canvas, selecție în listă, buton din toolbar, răspuns de la worker sau eveniment primit din colaborare.

A treia metodă relevantă este `_setup_shortcuts()`, care definește scurtături pentru acțiunile frecvente. `Delete` șterge adnotarea selectată, `Ctrl+O` deschide un folder de imagini, iar `Ctrl+S` salvează adnotările în format JSON. Deși sunt detalii mici, aceste scurtături arată că aplicația este gândită ca instrument de lucru repetitiv, nu doar ca prototip demonstrativ.

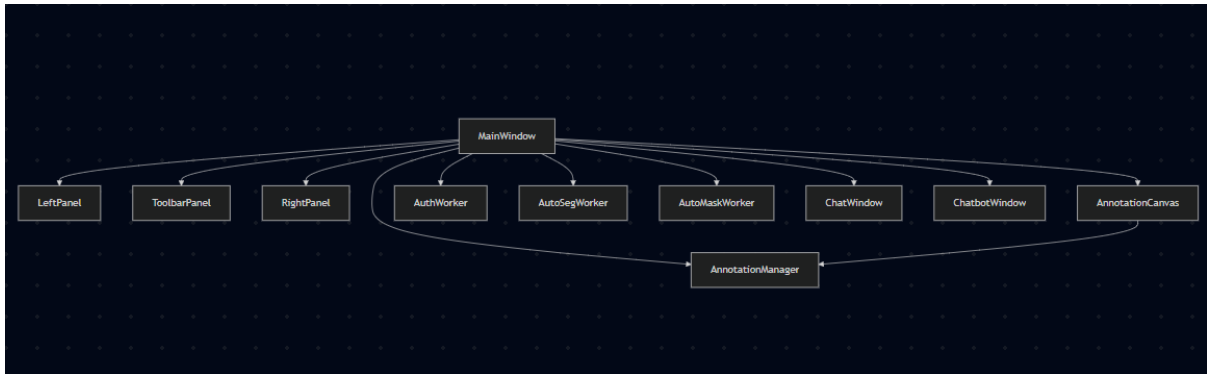


Figura 5.3: Organizarea internă a ferestrei principale

Figura 5.3 trebuie să evidențieze faptul că `MainWindow` nu este doar o fereastră grafică, ci un strat de orchestrare. În această clasă se păstrează stări precum `_unsaved_changes`, `_ignore_changes`, `_applying_remote_collab`, referințele către worker-ele active și referințele către ferestrele auxiliare. Aceste stări nu aparțin unui singur panou, deoarece influențează mai multe părți ale aplicației: salvarea, încărcarea, colaborarea, închiderea aplicației și prevenirea trimerii unor evenimente duplicate.

Pentru a evita transformarea clasei principale într-un fișier foarte greu de urmărit, o parte din fluxuri este extrasă în module din `utils/main_window_parts`. De exemplu, încărcarea imaginilor este tratată în `load_process.py`, starea de modificări nesalvate în `dirty_state.py`, autentificarea în `auth_handlers.py`, iar integrarea `AutoSeg` și `AutoMask` în fișiere dedicate. Această împărțire păstrează `MainWindow` ca punct de conectare, dar mută detaliile operaționale în module tematice.

5.2.3. Panourile aplicației

Interfața principală este împărțită în trei panouri și o bară de unelte. Fiecare panou are o responsabilitate clară, iar comunicarea cu restul aplicației se face prin semnale. Această structură reduce dependențele circulare și permite modificarea unui panou fără rescrierea întregii ferestre principale.

`LeftPanel` este panoul de navigare prin imagini. El permite deschiderea unui folder, filtrează fișierele după extensii de imagine precum `jpg`, `png`, `bmp`, `tiff` sau `webp` și afișează rezultatul într-o listă. Atunci când utilizatorul selectează o imagine, panoul emite `image_load_requested`. O funcție importantă a acestui panou este protecția la schimbarea imaginii atunci când există adnotări nesalvate. În această situație, utilizatorul poate salva, poate continua fără salvare sau poate anula schimbarea. Panoul nu face salvarea direct, ci cere ferestrei principale să o realizeze, deoarece toolbar-ul și managerul de adnotări dețin datele necesare.

`ToolbarPanel` concentrează acțiunile globale ale aplicației. De aici se pot salva adnotările în JSON, exporta datele în template COCO sau YOLO, salva o imagine cu măștile desenate, încărca un fișier JSON, deschide dialogul de cont, fereastra de chat, fereastra de chatbot și dialogul de setări. Toolbar-ul deține și logica de export, deoarece exportul este o acțiune globală asupra imaginii curente și asupra tuturor adnotărilor din `AnnotationManager`.

`SettingsDialog`, deschis din toolbar, este organizat pe taburi. Un tab controlează stilul vizual al adnotărilor, cum ar fi grosimea conturului, dimensiunea fontului și

înălțimea etichetei. Alt tab configurează AutoMask, prin calea către executabil, timeout, device și argumente suplimentare. Un tab separat configurează AutoSeg YOLO, prin executabil, fișierul de ponderi, pragul de confidență, device și argumente custom. Ultimul tab gestionează endpoint-urile pentru autentificare. Faptul că aceste valori sunt păstrate în `QSettings` permite aplicației să rețină configurarea între rulări.

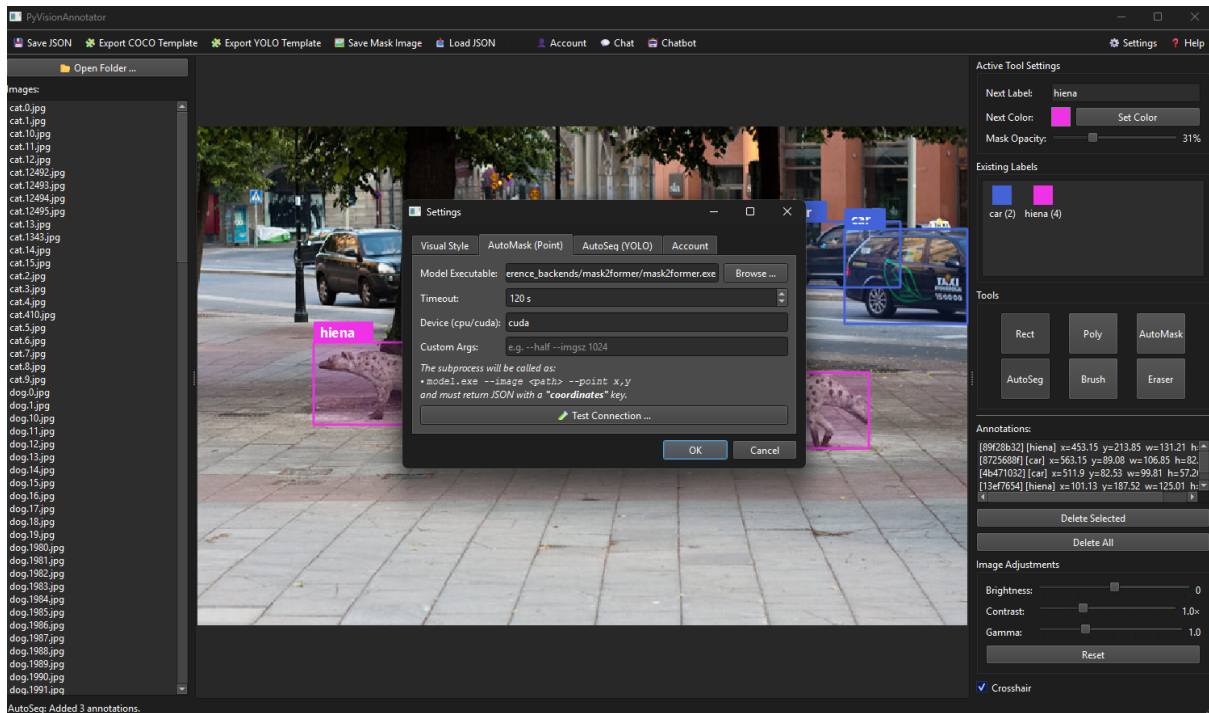


Figura 5.4: Panoul de setări pentru AutoSeg și AutoMask

`RightPanel` este panoul în care utilizatorul controlează adnotarea curentă și modul de lucru. El afișează proprietățile obiectului selectat, inclusiv identificatorul, eticheta, culoarea și coordonatele. Tot aici se află lista de adnotări, lista etichetelor existente, alegerea uneltei active și setările vizuale ale imaginii. Pentru măști există control de opacitate, iar pentru editare există brush și eraser. Panoul drept este conectat la `AnnotationManager`, astfel încât lista de adnotări și lista de etichete se actualizează atunci când se adaugă, modifică sau șterge un item.



Figura 5.5: Panoul de proprietăți și lista de adnotări

Un aspect important al panourilor este că ele nu sunt simple containere vizuale. Fiecare panou deține o parte din logica de interacțiune apropiată de interfață: **LeftPanel** decide când trebuie cerută salvarea înainte de schimbarea imaginii, **ToolBarPanel** decide cum se declanșează salvarea sau exportul, iar **RightPanel** decide cum se schimbă unealta activă și cum se actualizează proprietățile unei adnotări selectate. Totuși, operațiile care afectează sistemul în ansamblu sunt propagate către **MainWindow**, canvas sau manager, păstrând o delimitare practică între UI și starea aplicației.

5.2.4. Canvasul de adnotare

Canvasul este implementat prin clasa `AnnotationCanvas`, derivată din `QGraphicsView`. Această alegere permite folosirea unui `QGraphicsScene` în care imaginea, adnotările, marker-ele temporare și elementele de feedback pot fi reprezentate ca obiecte grafice separate. Imaginea încărcată este afișată ca `QGraphicsPixmapItem`, iar adnotările sunt adăugate peste ea cu valori de `Z` mai mari, astfel încât să rămână vizibile și selectabile.

```
self._scene = QGraphicsScene(self)
self.setScene(self._scene)
self.setTransformationAnchor(
    QGraphicsView.ViewportAnchor.AnchorUnderMouse
)
```

Coordonatele folosite în canvas sunt coordonate de scenă, iar scena este aliniată cu dimensiunile imaginii. Aceasta înseamnă că un punct din scenă corespunde direct unui pixel din imagine, indiferent de nivelul de zoom sau de poziția scroll-ului. Această decizie simplifică exportul și sincronizarea, deoarece adnotările pot fi salvate în coordonate ale imaginii originale, nu în coordonate dependente de interfață.

Canvasul gestionează interacțiuni specifice lucrului cu imagini: zoom cu roțița mouse-ului, pan cu butonul din mijloc sau cu `Space` și drag, desenare de dreptunghiuri, desenare de poligoane, editare de măști cu brush sau eraser și apelarea `AutoMask` prin click. Pentru ca fișierul `canvas.py` să nu devină greu de urmărit, evenimentele sunt extrase în `utils/canvas_helpers/canvas_events.py`, iar funcțiile de randare și ajustare vizuală sunt extrase în module separate. Clasa `AnnotationCanvas` rămâne astfel interfața publică a canvasului, în timp ce detaliile de mouse, tastatură și desenare sunt grupate pe fișiere tematice.

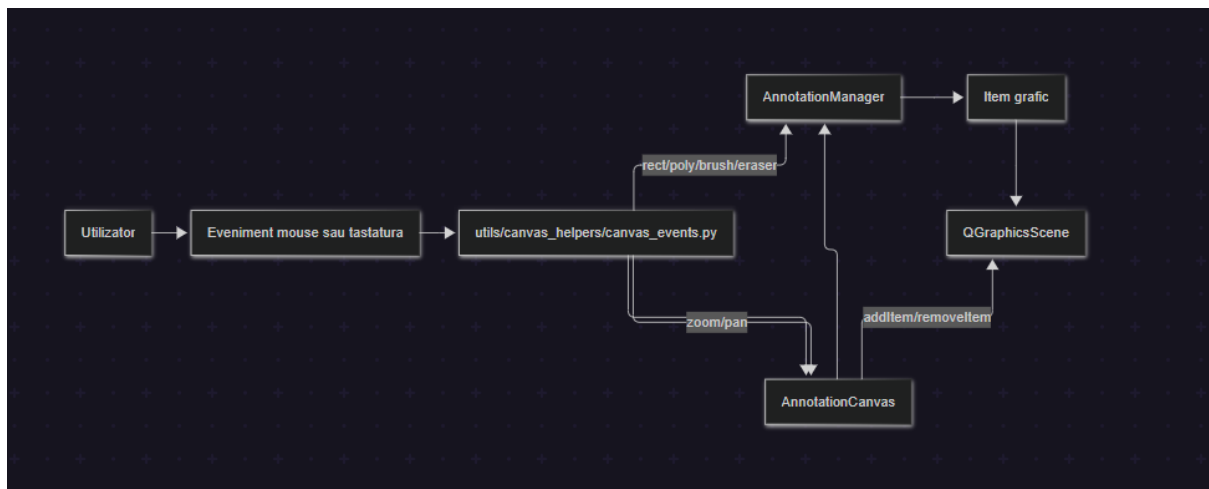


Figura 5.6: Fluxul de evenimente în canvas

Unealta de dreptunghi funcționează prin memorarea punctului de pornire, desena-rea temporară a unui `rubber band` și transformarea dreptunghiului final într-un `BoundingBoxItem`. Unealta de poligon adaugă puncte succesive în lista internă, afișează linii temporare și finalizează conturul la click dreapta, dublu click sau tasta `Enter`. Pentru brush și eraser, canvasul caută masca selectată și trimite poziția curentă către metodele

`paint_brush()` sau `erase_brush()` ale obiectului `MaskItem`. În cazul `AutoMask`, click-ul pe imagine nu creează direct o adnotare, ci emite semnalul `autoseg_requested`, care este preluat de fereastra principală și transformat într-un apel către worker.

Canvasul conține și ajustări vizuale pentru imagine: luminozitate, contrast și gamma. Aceste ajustări nu modifică fișierul original și nu schimbă coordonatele adnotărilor. Ele sunt aplicate doar pixmapului afișat, pe baza imaginii originale păstrate în memorie. Această diferență este importantă deoarece utilizatorul poate îmbunătăți vizibilitatea unei imagini întunecate sau slabe fără să altereze datele care vor fi exportate.

5.2.5. Obiectele grafice de adnotare

Adnotările sunt gestionate de `AnnotationManager`, care funcționează ca sursă locală de adevăr pentru imaginea curentă. Managerul păstrează un dicționar de obiecte grafice indexate după identificator și emite semnale atunci când o adnotare este adăugată, ștearsă, actualizată sau când lista este golită. Aceleași semnale sunt folosite pentru actualizarea panoului drept, marcarea modificărilor nesalvate și trimiterea evenimentelor de colaborare.

```
def get_all_dicts(self) -> list[dict]:
    return [item.to_dict() for item in self._annotations.values()]
```

Fragmentul arată una dintre ideile centrale ale implementării: adnotările sunt obiecte grafice, dar pot fi transformate în dicționare serializabile. Acest lucru permite folosirea aceluiași model intern atât pentru desenare pe canvas, cât și pentru salvare, export, colaborare și încărcare din fișiere.

Pentru bounding box-uri se folosește clasa `BoundingBoxItem`, derivată din `QGraphicsRectItem`. Aceasta desenează dreptunghiul, fundalul semitransparent, eticheta și mânerul de redimensionare. Atunci când un dreptunghi este selectat, utilizatorul poate modifica dimensiunea prin cele opt handle-uri. La finalul modificării geometriei, itemul notifică managerul, iar restul interfeței se actualizează prin semnale.

Pentru adnotările poligonale se folosește `PolygonItem`, derivată din `QGraphicsPolygonItem`. Aceasta reprezintă contururi definite prin puncte și are o implementare mai simplă decât dreptunghiul redimensionabil. Poligonul este potrivit pentru adnotări cu formă neregulată, dar cu un număr relativ redus de vârfuri. Ca și celelalte tipuri de item, el are etichetă, culoare, stil vizual și metodă `to_dict()`.

Pentru măști se folosește `MaskItem`, derivată din `QGraphicsPixmapItem`. Această clasă este diferită de celelalte deoarece o mască poate conține foarte multe puncte. Pentru randare eficientă, masca este transformată într-o imagine internă, colorată cu opacitate configurabilă, iar brush-ul și eraser-ul modifică direct pixmapul asociat. Această abordare reduce costul de desenare pentru măști dense și permite utilizatorului să le editeze fluent.

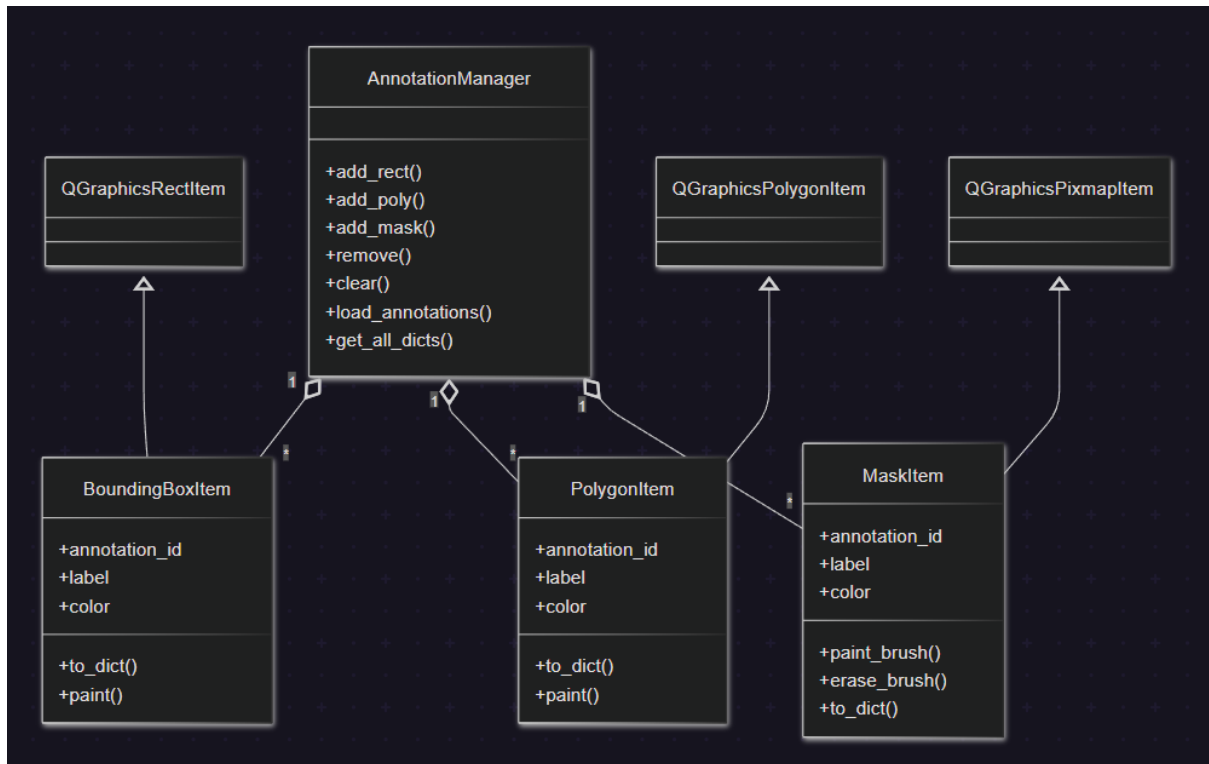


Figura 5.7: Clasele principale pentru adnotări

Figura 5.7 poate fi transformată într-o diagramă de clase simplificată. Nu este necesară listarea tuturor metodelor, deoarece scopul este să se vadă separarea dintre manager și tipurile concrete de adnotări. Detaliile complete despre serializare și export pot fi dezvoltate în secțiunile următoare, însă introducerea acestor clase în 5.2 ajută la înțelegerea modului în care interfața și datele sunt legate.

5.2.6. Integrarea AutoSeg YOLO și AutoMask în client

Modelele de segmentare sunt integrate în client prin fluxuri de interfață, nu prin apeluri directe în canvas. Pentru AutoSeg, utilizatorul pornește operația din panoul drept prin butonul **AutoSeg**. Handler-ul `on_autoseg_yolo_run()` citește setările din `QSettings`, verifică existența imaginii curente, deschide dialogul `AutoSegRunDialog` pentru etichete și moduri de ieșire, apoi pornește `AutoSegWorker`. Rezultatul poate conține `box`, `coordinates` sau `mask_coords`, iar fiecare variantă este convertită într-un tip intern: dreptunghi, poligon sau mască.

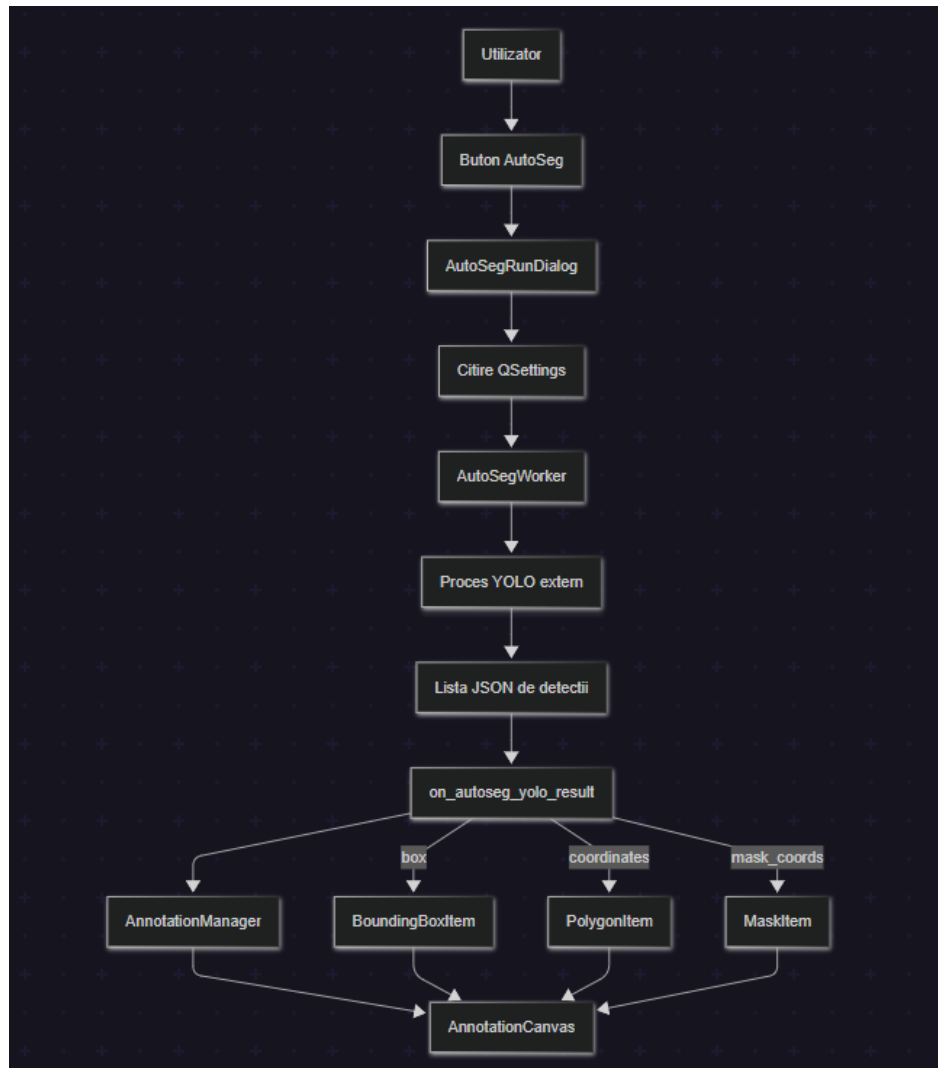


Figura 5.8: Fluxul AutoSeg YOLO în client

Pentru AutoMask, fluxul pornește din canvas. Utilizatorul selectează unealta AutoMask, apoi face click pe obiectul de interes. Canvasul adaugă un marker vizual temporar și emite semnalul `autoseg_requested` cu poziția în coordonate de scenă. Handler-ul citește executabilul, timeout-ul, device-ul și argumentele suplimentare, apoi pornește `AutoMaskWorker`. Dacă modelul returnează coordonate valide, acestea sunt transformate într-o mască și adăugate pe canvas.

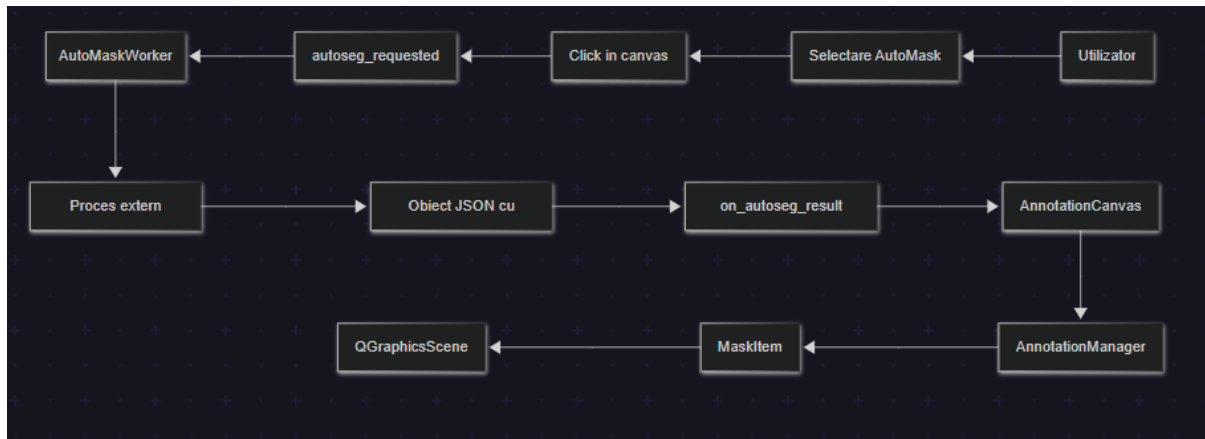


Figura 5.9: Fluxul AutoMask pe baza unui punct selectat

Ambele fluxuri folosesc `QProgressDialog`, astfel încât utilizatorul vede că procesarea este în curs și poate anula operația. Anularea nu se face prin închiderea brutală a aplicației sau prin blocarea firului principal, ci prin metoda `cancel()` a worker-ului, care ajunge la procesul gestionat intern. Această abordare este importantă pentru modelele AI, deoarece inferența poate dura mai mult decât o acțiune obișnuită de interfață.

Logica folosită în zona de cercetare arată că modelul concret nu este partea esențială a integrării, ci contractul de intrare și ieșire respectat de procesul extern. În `research/others/yoloe.py` scriptul primește argumente precum `-image`, `-labels`, `-conf`, `-device` și `-mode`. După încărcarea modelului YOLOE și setarea claselor cerute, scriptul rulează predicția și scrie în `stdout` o listă JSON de detecții. Fiecare detecție poate conține eticheta, scorul de încredere, coordonatele dreptunghiului și coordonatele segmentării.

```
[
  {
    "label": "car",
    "confidence": 0.87,
    "box": [120.0, 80.0, 340.0, 250.0],
    "coordinates": [[130.0, 90.0], [330.0, 95.0], [320.0, 240.0]]
  }
]
```

`AutoSegWorker` așteaptă ca rezultatul procesului să poată fi interpretat ca listă JSON. Dacă procesul întoarce un singur obiect JSON, worker-ul îl transformă într-o listă cu un singur element, dar handler-ul principal lucrează tot cu o listă de detecții. În `on_autoseg_yolo_result()`, câmpul `box` este convertit într-un `BoundingBoxItem`, iar câmpul `coordinates` este convertit într-un `PolygonItem`. Există și suport pentru cheia `mask_coords`, care poate fi transformată direct într-un `MaskItem`. Astfel, `AutoSeg` nu depinde strict de o anumită implementare YOLO, ci de faptul că procesul extern returnează câmpuri pe care clientul le poate transforma în adnotări.

În mod similar, `research/others/mask2former.py` implementează un flux de segmentare pe baza unui punct. Scriptul primește `-image`, `-point`, `-model` și `-device`, verifică dacă punctul se află în imagine, rulează `Mask2Former` și identifică segmentul pe care se află punctul selectat. Rezultatul este un obiect JSON care conține eticheta seg-

mentului, identificatorul intern, punctul interogată, coordonatele pixelilor segmentului și un dreptunghi de încadrare.

```
{
  "label": "car",
  "id": 12,
  "point_queried": [180, 140],
  "coordinates": [[130, 90], [131, 90], [132, 91]],
  "bbox": [120, 80, 340, 250]
}
```

AutoMaskWorker verifică în mod explicit existența cheii **coordinates**; fără aceasta, rezultatul este considerat invalid pentru client. Handler-ul **on_autoseg_result()** transmite datele către canvas, iar canvasul creează un **MaskItem** din coordonatele primite. Cheile suplimentare, precum **label**, **id**, **point_queried** sau **bbox**, pot fi utile pentru diagnosticare sau extensii, dar cheia necesară pentru crearea măștii este **coordinates**.

Prin urmare, **YOLOE** și **Mask2Former** sunt exemple concrete de modele folosite în dezvoltare, însă arhitectura clientului permite înlocuirea lor cu alte executabile sau scripturi. Condiția este ca acestea să accepte parametrii transmiși de worker-e și să emită în **stdout** JSON în forma așteptată de **AnnotationEngine**. Această separare face ca aplicația să fie mai flexibilă: modelul poate evolua separat, iar interfața rămâne responsabilă doar de configurare, execuție, parsare și transformarea rezultatului în adnotări editabile.

5.2.7. Worker-e și procese externe

Pentru a păstra interfața responsivă, operațiile costisitoare nu sunt executate în firul principal al aplicației. Clientul folosește mai multe worker-e bazate pe **QThread**: **AutoSegWorker** pentru segmentarea **YOLO**, **AutoMaskWorker** pentru segmentarea pe punct sau pe întreaga imagine, **AuthWorker** pentru autentificare, **ChatStompWorker** pentru chat, **CollabStompWorker** pentru colaborare, **OllamaChatWorker** și **LlamaChatWorker** pentru chatbot local. Fiecare worker expune semnale de succes, eroare sau rezultat intermediar, iar UI-ul reacționează la acestea.

O parte importantă a acestei structuri este **SubprocessHandler**. El este folosit pentru executarea proceselor externe asociate modelelor AI. Comenzile sunt construite ca liste de argumente, nu ca string-uri interpretate de shell. Prin folosirea **shell=False**, aplicația reduce riscul de injecție de comenzi și controlează mai bine argumentele transmise executabilului. Handler-ul verifică existența executabilului sau a scriptului, pornește procesul, colectează **stdout** și **stderr**, aplică timeout și încearcă să extragă JSON din ieșirea procesului.

```
proc = subprocess.Popen(
    command,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
    shell=False,
)
```

Pentru anulare, `SubprocessHandler` folosește o clasă `ManagedProcess`, care învelește procesul pornit și expune metoda `kill()`. Worker-ul păstrează o referință la acest proces și îl poate opri atunci când utilizatorul apasă butonul **Cancel** din dialogul de progres. După terminare, worker-ul emite fie rezultatul parsabil, fie un mesaj de eroare care este afișat în interfață.

Aceeași idee se aplică și comunicării cu serviciile locale sau backend. Worker-ele pentru Ollama și llama.cpp trimit cereri de chat fără să blocheze fereastra de chatbot. Worker-ele STOMP gestionează bucla de recepție WebSocket în fundal, iar evenimentele primite sunt aduse în interfață prin semnale Qt. Prin această structură, aplicația poate rula simultan operații de interfață, inferență AI și comunicare în timp real fără ca utilizatorul să simtă blocaje majore.

5.2.8. Ferestre auxiliare: chat, colaborare și chatbot local

Pe lângă fereastra principală, clientul poate deschide ferestre auxiliare pentru comunicare și asistență locală. `ChatWindow` este lansată din toolbar și este responsabilă pentru chatul în timp real și funcțiile colaborative. Ea nu este implementată integral într-un singur fișier, ci folosește module separate pentru construirea UI-ului, conexiune, mesaje, setări și colaborare. Această împărțire urmează aceeași idee ca la `MainWindow`: clasa principală păstrează starea și legăturile, iar detaliile operaționale sunt mutate în fișiere tematice.

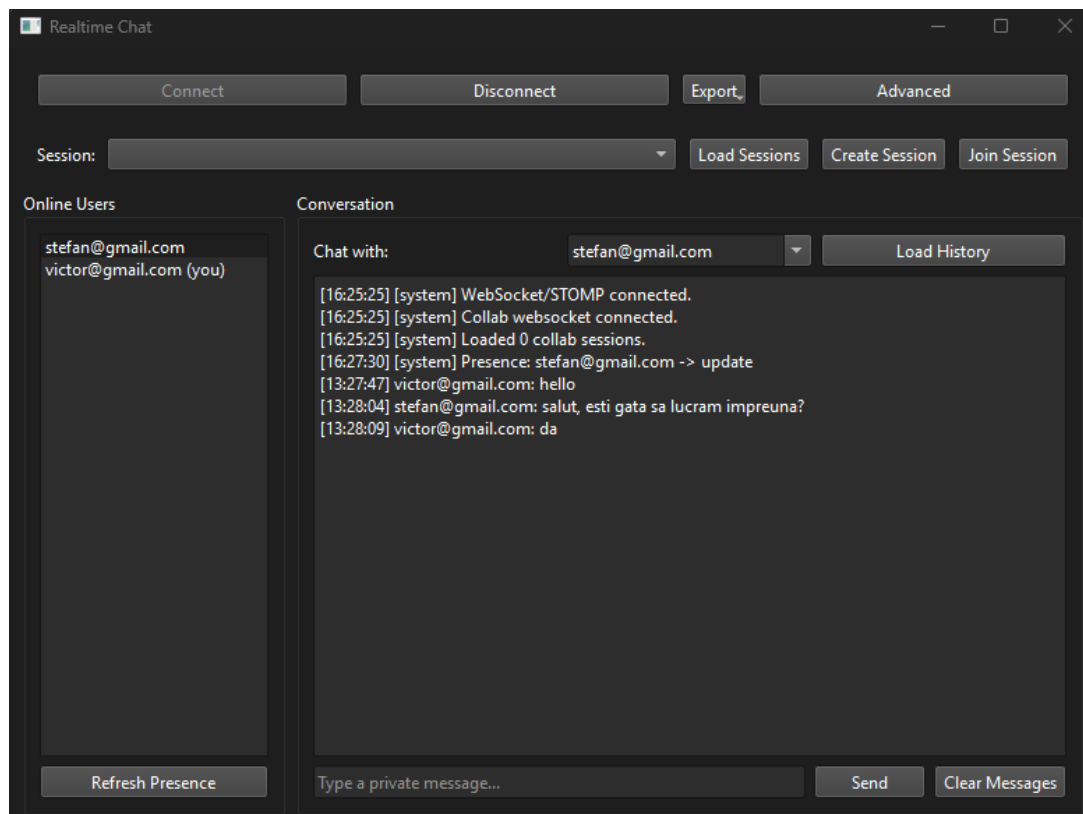


Figura 5.10: Fereastra de chat și colaborare

Pentru colaborare, `ChatWindow` se conectează la backend prin worker-e STOMP și printr-un client REST pentru operații precum listarea sesiunilor, crearea unei sesiuni sau

descărcarea unei imagini partajate. Evenimentele de tip `annotation.create`, `annotation.update` și `annotation.delete` ajung înapoi în `MainWindow`, unde sunt normalizate și aplicate pe `AnnotationManager`. În sens invers, modificările locale ale adnotărilor pot fi transformate în payload-uri colaborative și trimise către sesiunea activă.

`ChatbotWindow` oferă o interfață pentru modele conversaționale locale. Utilizatorul poate alege între Ollama și llama.cpp, poate selecta sau încărca modelul dorit și poate trimite mesaje. Pentru Ollama există un worker care listează modelele disponibile și un worker care trimite cererea de chat cu streaming de tokeni. Pentru llama.cpp, fereastra permite configurarea unui fișier GGUF și a unor parametri precum contextul, temperatura sau numărul maxim de tokeni. Răspunsurile sunt afișate incremental, pe măsură ce worker-ul emite tokeni.

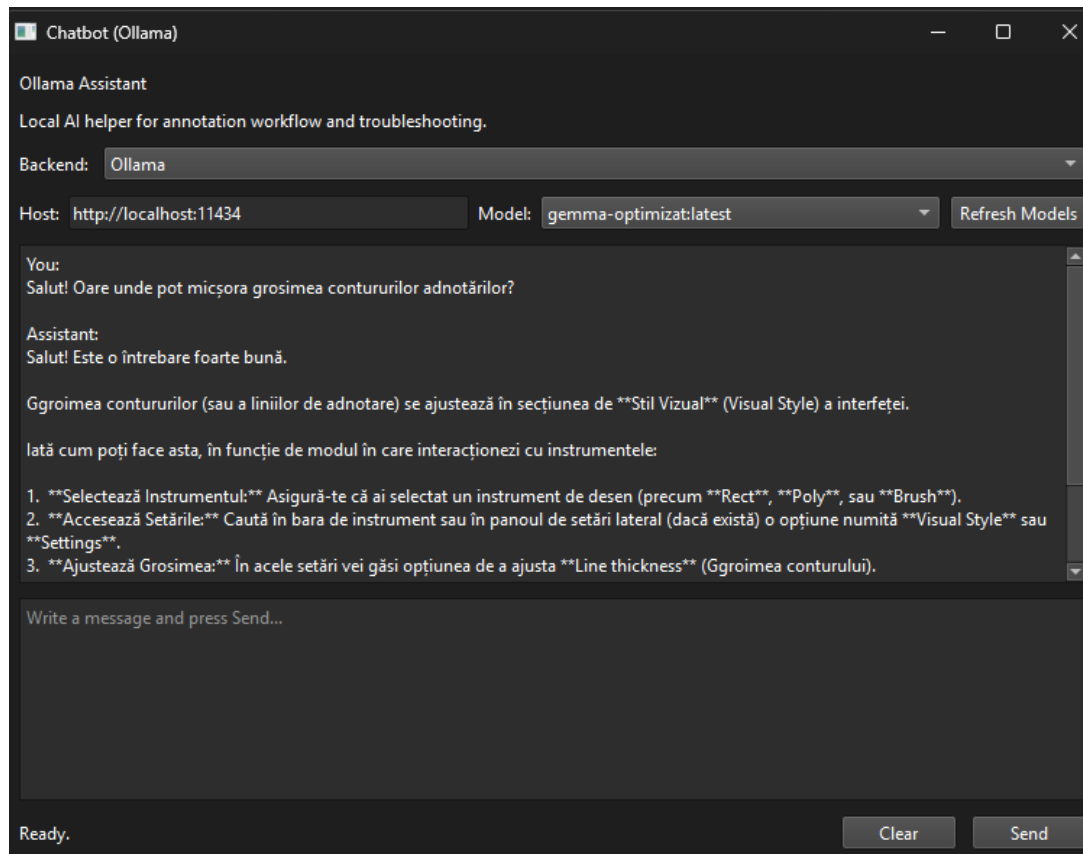


Figura 5.11: Fereastra chatbot local cu Ollama sau llama.cpp

Aceste ferestre auxiliare arată că aplicația desktop nu este doar un editor de forme geometrice. Ea este punctul de intrare într-un sistem mai mare, în care utilizatorul poate cere ajutor unui LLM local, poate comunica cu alți utilizatori și poate sincroniza adnotări într-o sesiune comună. Totuși, integrarea lor este făcută astfel încât fereastra principală să rămână utilizabilă și când aceste funcții nu sunt deschise.

5.2.9. Gestionarea stării locale

O aplicație de adnotare trebuie să protejeze munca utilizatorului. În client, acest lucru este realizat printr-o stare locală de tip `_unsaved_changes`, care marchează dacă imaginea curentă are modificări nesalvate. Semnalele din `AnnotationManager` setează

această stare atunci când se adaugă, se șterge sau se actualizează o adnotare. În anumite situații, precum încărcarea unui JSON sau aplicarea unui snapshot colaborativ, aplicația setează `_ignore_changes`, pentru ca operațiile programatice să nu fie interpretate ca modificări manuale ale utilizatorului.

Fluxul de schimbare a imaginii este un exemplu bun pentru această logică. Atunci când utilizatorul selectează alt fișier în `LeftPanel`, panoul verifică dacă există modificări nesalvate. Dacă nu există, emite direct cererea de încărcare. Dacă există, utilizatorul este întrebat dacă dorește să salveze înainte de schimbare. Dacă alege salvarea, panoul nu salvează singur, ci emite `save_before_switch_requested`; `MainWindow` apelează salvarea din toolbar, iar apoi confirmă sau anulează schimbarea imaginii.

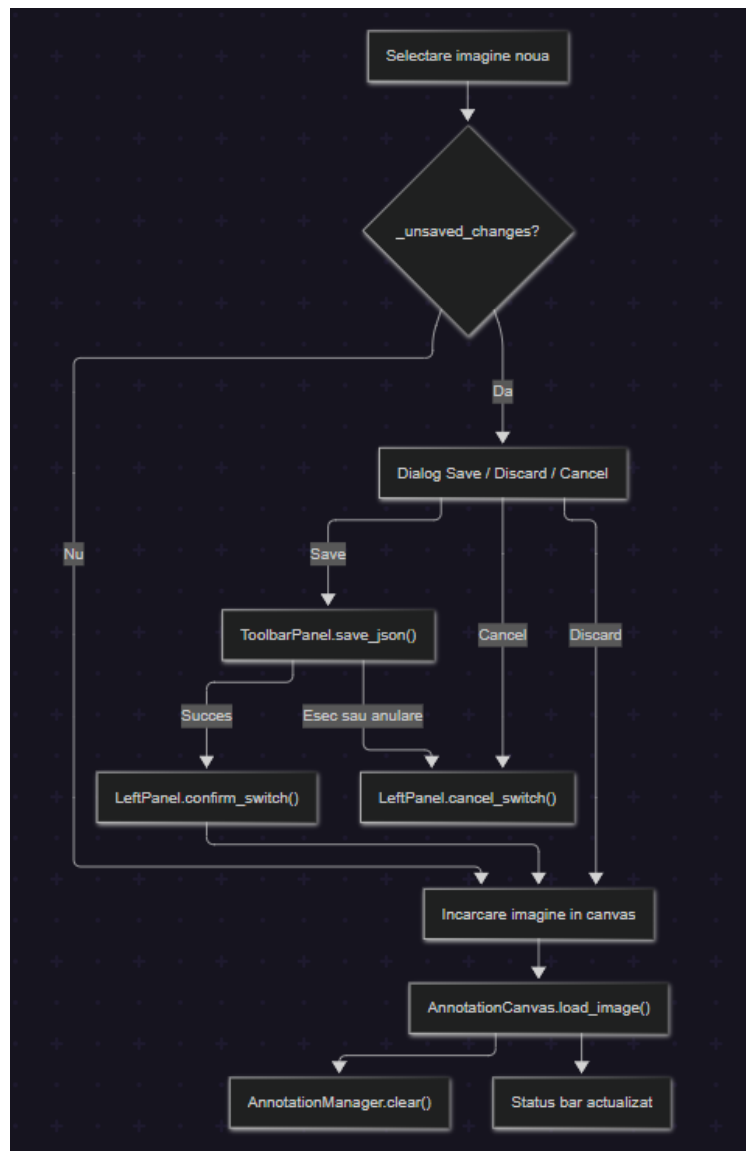


Figura 5.12: Fluxul de încărcare imagine și protecția modificărilor nesalvate

Fișierul `load_process.py` grupează operațiile de încărcare a unei imagini, încărcare a adnotărilor din JSON și aplicare a setărilor vizuale. Când se importă un fișier JSON, aplicația poate încărca imaginea asociată, poate goli adnotările existente și poate reconstrui obiectele grafice prin `AnnotationManager.load_annotations()`. Fișierul `dirty_state.py`

tratează marcarea modificărilor, resetarea stării după salvare și protecția la închiderea aplicației.

Gestionarea stării locale este importantă și pentru colaborare. Atunci când aplicația primește o actualizare de la server, flag-ul `_applying_remote_collab` previne retrimite-rea aceleiași modificări ca eveniment local. Pentru măști, unde editarea cu brush poate genera multe modificări într-un timp scurt, aplicația folosește un timer pentru a grupa update-urile înainte de trimitere. Astfel, clientul reduce traficul WebSocket și evită publicarea excesivă de evenimente pentru fiecare mișcare mică a mouse-ului.

Prin aceste mecanisme, clientul desktop este structurat ca o aplicație de lucru completă: are o interfață împărțită clar, un canvas interactiv, obiecte de adnotare serializabile, worker-e pentru sarcini costisitoare, ferestre auxiliare pentru colaborare și o stare locală care protejează datele utilizatorului. Această structură permite extinderea ulterioară a aplicației, de exemplu prin adăugarea unor noi tipuri de export, noi modele de segmentare sau noi instrumente de editare, fără a schimba fundamental arhitectura clientului.

5.3. Autentificare și model de acces

Partea de backend începe cu serviciul de autentificare, deoarece funcțiile de chat și colaborare au nevoie de o identitate stabilă a utilizatorului. În proiect, această responsabilitate este izolată în modulul `auth_module`, rulat în Docker ca `auth-service`. Serviciul este o aplicație Spring Boot care expune endpoint-uri REST sub prefixul `/auth` și folosește Postgres pentru stocarea conturilor. Clientul desktop nu accesează direct baza de date, ci comunică prin cereri HTTP, iar rezultatul autentificării este folosit apoi pentru restul serviciilor.

Controllerul principal este `AuthenticationController`. Acesta definește patru operații importante: `/auth/login`, `/auth/register`, `/auth/logout` și `/auth/validate`. Primele două sunt apelate direct de aplicația desktop prin `AuthWorker`, care trimite emailul și parola într-un corp JSON și așteaptă un răspuns de tip `AuthResponse`. Endpoint-ul de validare are un rol diferit: el este folosit și de Traefik ca mecanism de `forwardAuth`, astfel încât serviciile de chat și annotation să poată fi protejate fără să implementeze fiecare aceeași logică de login.

```
POST /auth/login
POST /auth/register
POST /auth/logout
GET /auth/validate
```

La înregistrare, `UserService` verifică dacă emailul există deja, criptează parola prin `PasswordEncoder` și salvează utilizatorul în tabela gestionată de repository-ul JPA. Răspunsul nu întoarce parola, ci o variantă sigură a obiectului `User`, care conține identificatorul și emailul. Această separare este importantă deoarece aplicația desktop are nevoie doar de identitatea utilizatorului, nu de detalii interne despre credențiale.

La autentificare, `UserService.login()` folosește `AuthenticationManager` pentru verificarea parolei, iar apoi generează un token JWT prin `JWTUtils`. În implementarea curentă, token-ul are ca subiect emailul utilizatorului, include un claim `roles` și este semnat cu o cheie HMAC SHA-256 configurată prin variabila `JWT_SECRET`. Durata de

expirare este definită în cod la 24 de ore. Structura generală a unui token JWT a fost prezentată anterior în Figura 4.4; în capitolul de implementare este important mai ales modul în care token-ul circulă între componente.

După login, clientul salvează payload-ul primit local, prin `utils/auth_store.py`. Salvarea se face în directorul de aplicație furnizat de `QStandardPaths`, într-un fișier JSON asociat sesiunii curente. În acest mod, fereastra de chat și fluxurile de colaborare pot reutiliza token-ul fără ca utilizatorul să introducă datele de autentificare la fiecare operație. Token-ul este apoi trimis în header-ul `Authorization: Bearer ...` pentru cererile REST și pentru conexiunile WebSocket/STOMP.

Endpoint-ul `/auth/validate` extrage token-ul din header, verifică forma `Bearer`, citește subiectul cu `JWTUtils.extractUsername()` și respinge token-urile expirate sau invalide. Dacă token-ul este valid, răspunsul conține header-ul `X-User-Id`. Acest header este relevant pentru serviciile aflate în spatele proxy-ului: Traefik poate valida cererea la `auth-service`, iar serviciul țintă poate primi identitatea utilizatorului fără să refacă întregul flux de autentificare.

Logout-ul este tratat stateless. Endpoint-ul `/auth/logout` extrage numele utilizatorului din token și apelează `UserService.logout()`, dar implementarea nu menține o listă de token-uri revocate. Această decizie este coerentă cu folosirea JWT în forma sa simplă: serverul nu păstrează sesiuni active pentru fiecare login, iar valabilitatea token-ului este controlată prin semnătură și timp de expirare. Pentru o variantă de producție s-ar putea adăuga o listă de revocare sau refresh token-uri, dar pentru proiectul curent fluxul este suficient pentru protejarea serviciilor colaborative.

Un detaliu important este publicarea evenimentului de înregistrare în RabbitMQ. După salvarea unui utilizator nou, `UserService` construiește un `EventMessage` cu emailul, timestamp-ul și tipul `register`, îl serializează JSON și îl publică în exchange-ul `auth.events`, cu routing key-ul `auth.user.register`. Serviciile care au nevoie să cunoască utilizatorii, cum este chatul, se pot sincroniza ascultând acest eveniment. Astfel, autentificarea rămâne sursa principală pentru conturi, iar celelalte module nu trebuie să acceseze direct baza Postgres de auth.

Din perspectiva aplicației desktop, fluxul complet este simplu. Utilizatorul deschide dialogul de cont, introduce emailul și parola, `AuthWorker` trimite cererea către endpoint-ul configurat, iar la succes payload-ul este salvat local. Ulterior, când utilizatorul deschide chatul sau colaborarea, același token este atașat la conexiunile REST și WebSocket. Astfel, autentificarea este integrată în aplicație fără să blocheze interfața și fără să amestece logica de cont cu logica de adnotare.

5.4. Chat în timp real și prezență

Serviciul de chat este separat de serviciul de annotation. Această separare există deoarece mesajele private, prezența utilizatorilor și istoricul conversațiilor au un model diferit față de sincronizarea adnotărilor. În `chat_module`, comunicarea se face prin WebSocket peste STOMP, iar controllerul principal este `ChatController`. Clientul desktop folosește `ChatStompWorker`, un `QThread` care deschide conexiunea WebSocket, trimite frame-ul STOMP `CONNECT`, face subscribe la destinațiile relevante și procesează mesajele primite în fundal.

Endpoint-ul WebSocket al serviciului este `/ws`. În `WebSocketConfig`, aplicația

configurează brokerul simplu pentru prefixele `/topic` și `/queue`, prefixul de aplicație `/app` și prefixul de user `/user`. Configurația înregistrează același endpoint `/ws` atât cu SockJS, cât și fără SockJS. În clientul desktop se folosește conexiunea WebSocket directă, dar existența variantei SockJS permite fallback pentru clienți care nu pot folosi WebSocket nativ.

```
SEND      /app/chat.send
SEND      /app/chat.history
SEND      /app/chat.presence
SUBSCRIBE /user/queue/private
SUBSCRIBE /user/queue/history
SUBSCRIBE /user/queue/presence
SUBSCRIBE /topic/presence
```

Trimiterea unui mesaj privat începe în client prin `ChatStompWorker.send_private()`. Worker-ul pune comanda într-o coadă internă, iar bucla de execuție o transformă într-un frame STOMP cu destinația `/app/chat.send`. Serverul primește mesajul în `ChatController.sendPrivateMessage()`, extrage utilizatorul curent din `Principal` și apelează `ChatService.sendPrivateMessage()`. Serviciul normalizează emailurile, verifică faptul că destinatarul există și respinge mesajele goale sau conversațiile directe cu sine.

Istoricul este păstrat în memorie, într-un `ConcurrentHashMap` care mapează cheia conversației la o coadă de mesaje. Cheia conversației este construită ordonat din cele două emailuri, astfel încât conversația dintre A și B să fie aceeași indiferent cine trimite mesajul. Pentru fiecare conversație se păstrează cel mult `MAX_MESSAGES_PER_CONVERSATION`, valoare definită la 200. Când limita este depășită, mesajele cele mai vechi sunt eliminate. Această alegere reduce memoria folosită de serviciu și este potrivită pentru un backend de colaborare live, unde exportul final al datasetului nu depinde de istoricul complet de chat.

După validare, același mesaj este trimis prin `SimpMessagingTemplate` atât către expeditor, cât și către destinatar, pe coada `/queue/private`. Din perspectiva clientului, acest lucru înseamnă că fereastra de chat primește și mesajele trimise local, nu doar pe cele venite de la celălalt utilizator. Astfel, interfața poate afișa o singură sursă de evenimente și nu trebuie să aibă o ramură separată pentru mesajele proprii.

Prezența este gestionată prin `PresenceService` și `WebSocketPresenceListener`. La conectarea unei sesiuni WebSocket, utilizatorul este marcat online, iar la deconectare este marcat offline. Evenimentele de prezență sunt publicate pe `/topic/presence`, iar snapshot-ul cerut explicit de client este trimis pe `/queue/presence`. În acest mod, fereastra de chat poate afișa utilizatorii disponibili și poate actualiza starea lor fără polling HTTP repetat.

Sincronizarea utilizatorilor dintre auth și chat se face prin RabbitMQ. `UserSyncListener` ascultă coada `chat.user.register.queue`, parsează evenimentul de tip `register`, normalizează emailul și creează sau actualizează utilizatorul în baza H2 a modulului de chat. Această bază nu este sursa principală de autentificare, ci o copie locală necesară pentru verificarea destinatarilor și afișarea unui nume derivat din email. Prin această soluție, chatul rămâne decuplat de Postgres-ul serviciului de auth, dar poate totuși valida că un destinatar există.

Din punct de vedere funcțional, chatul este un canal de comunicare între utilizatori,

nu mecanismul prin care se sincronizează adnotările. Această delimitare este importantă în documentație: mesajele private sunt tratate ca text și istoric de conversație, în timp ce colaborarea pe imagine este tratată ca stare partajată, versionată și aplicată pe canvas. Ambele folosesc WebSocket/STOMP, dar modelele de date și regulile de aplicare sunt diferite.

5.5. Colaborarea pe adnotări

Colaborarea pe adnotări este implementată în `annotation_module`. Acest serviciu combină endpoint-uri REST pentru inițializare cu WebSocket/STOMP pentru sincronizare live. REST-ul este folosit pentru operații care au un răspuns clar și punctual, precum listarea sesiunilor, crearea unei sesiuni, obținerea unui snapshot sau încărcarea temporară a imaginii. WebSocket-ul este folosit pentru evenimente care trebuie propagate rapid către toți participanții unei sesiuni: intrarea unui utilizator, părăsirea sesiunii, disponibilitatea imaginii și modificările de adnotare.

Din exterior, prin Traefik, endpoint-urile sunt accesate sub prefixul `/annotation`. În interiorul containerului, prefixul este eliminat prin middleware-ul `annotation-strip-prefix`, iar serviciul vede rutele simple `/sessions`, `/media` și `/ws`. Această diferență explică de ce în client apar constante precum `/annotation/sessions` și `/annotation/ws`, în timp ce controller-ele Spring sunt mapate la `/sessions`, respectiv `/media`.

```
GET /annotation/sessions
POST /annotation/sessions
GET /annotation/sessions/{sessionId}/snapshot
POST /annotation/media/upload-temp
GET /annotation/media/temp/{imageId}?token=...
WS /annotation/ws
```

`SessionController` gestionează sesiunile colaborative. La `GET /sessions`, serviciul întoarce lista camerelor active. La `POST /sessions`, creează o sesiune nouă, generează un cod scurt de cinci caractere și marchează utilizatorul curent ca membru online. După creare, serverul publică și un eveniment `session.created` pe `/topic/sessions`, pentru ca alți clienți conectați să poată actualiza lista de sesiuni fără refresh manual.

Snapshot-ul este mecanismul prin care un client intră într-o stare consistentă înainte de a consuma evenimente live. Când utilizatorul se alătură unei sesiuni, clientul cere `/sessions/{sessionId}/snapshot`. Serverul verifică sau creează apartenența utilizatorului, apoi întoarce un obiect care include identificatorii sesiunii, utilizatorii, adnotările curente, metadatele imaginii și versiunea curentă. Clientul aplică snapshot-ul local, setează ultima versiune cunoscută și apoi acceptă evenimentele noi venite pe topicul sesiunii.

`MediaController` rezolvă problema imaginii partajate. Atunci când un utilizator creează sau folosește o sesiune, imaginea activă poate fi încărcată temporar prin `/media/upload-temp`. Serverul verifică dacă fișierul este o imagine validă, îi calculează dimensiunile, dimensiunea în bytes și un checksum SHA-256, apoi salvează conținutul în `TempImageStore`. Răspunsul este un `ImageMetaDto` care conține și un `downloadUrl` cu token temporar. Ceilalți clienți pot descărca imaginea prin `/media/temp/{imageId}`, fără ca aceasta să devină parte din datasetul final exportat.

Evenimentele live sunt tratate de `CollaborationWsController`, prin destinația STOMP `/app/collab.event`. Clientul se abonează la `/topic/sessions` pentru lista

globală de sesiuni și la `/topic/sessions/{sessionId}` pentru evenimentele camerei active. Fiecare eveniment este încapsulat într-un `WsEventEnvelope`, structură comună pentru tipul evenimentului, actor, sesiune, versiune și payload.

```
{
  "eventId": "...",
  "type": "annotation.update",
  "sessionId": "...",
  "projectId": "...",
  "imageId": "...",
  "cameraId": "...",
  "actorId": "...",
  "version": 12,
  "timestamp": "...",
  "payload": { }
}
```

Pentru evenimentele `session.user.joined` și `session.user.left`, controllerul actualizează prezența utilizatorului în sesiune și publică rezultatul către topicul camerei. Pentru `image.available`, controllerul construiește un eveniment cu metadatele imaginii și îl trimite tuturor participanților. Pentru evenimentele de adnotare, controllerul validează apartenența utilizatorului la sesiune și apelează `CollaborationService.applyAnnotationEvent()`.

`CollaborationService` este sursa de adevăr pentru starea colaborativă a sesiunilor active. Starea este păstrată în memorie, într-un `ConcurrentHashMap` numit `session-sById`. Fiecare sesiune conține metadatele proiectului, utilizatorii, adnotările și versiunea curentă. La `annotation.create` și `annotation.update`, payload-ul este validat și salvat în mapa de adnotări a sesiunii. La `annotation.delete`, adnotarea este eliminată după identificator. După aplicare, versiunea sesiunii este incrementată și evenimentul normalizat este publicat către clienți.

Validarea este mai strictă pentru măști, deoarece o mască fără puncte nu poate fi aplicată pe canvas. În cazul unui payload de tip `mask`, serviciul caută punctele fie în `payload.points`, fie în `payload.mask.points`, verifică existența listei și validează fiecare punct ca pereche numerică finită `[x, y]`. Această validare previne propagarea unor măști incomplete care ar putea rupe randarea pe client.

Deduplicarea este gestionată de `EventDedupStore`. Fiecare `eventId` procesat este păstrat pentru o perioadă configurabilă, implicit `PT10M`. Dacă același eveniment ajunge din nou la server, serviciul îl consideră duplicat și nu îl mai aplică. Pe client există o logică similară: `CollabStompWorker` păstrează o listă limitată de evenimente văzute, iar `chat_window_collab.py` păstrează ultimele versiuni primite pe sesiune. Cele două mecanisme reduc riscul de aplicare repetată a aceleiași modificări după reconectări sau retry-uri.

Prin această arhitectură, colaborarea este construită ca un flux de evenimente versionate. REST-ul aduce clientul într-o stare de pornire, iar WebSocket-ul livrează diferențele apărute după aceea. Clientul nu trebuie să descarce tot proiectul la fiecare modificare, iar serverul nu trebuie să salveze permanent datasetul final. Sesiunea live rămâne în memorie, iar exportul final rămâne responsabilitatea aplicației desktop.

5.5.1. Trimiterea adnotărilor mari în mai multe pachete

Măștile pot conține foarte multe puncte, mai ales când rezultatul vine dintr-un model de segmentare sau când utilizatorul editează zona cu brush-ul. Dacă un astfel de payload ar fi trimis într-un singur frame WebSocket, există riscul ca mesajul să fie prea mare pentru client, server sau proxy. Din acest motiv, aplicația implementează un mecanism de trimitere în mai multe pachete, denumite în cod `chunk-uri`.

Mecanismul pornește în client, în modulul de colaborare al ferestrei de chat. Funcția care decide ruta de trimitere verifică evenimentele de tip `annotation.create`, `annotation.update` și `annotation.delete`. Pentru măști create sau actualizate, evenimentul este trimis întotdeauna chunked. Pentru celelalte adnotări, chunking-ul se activează doar dacă envelope-ul serializat depășește pragul de $2 * 1024$ bytes. Dimensiunea fiecărui segment base64 este $8 * 1024$ caractere, iar numărul maxim de reîncercări este 2.

Pașii sunt următorii. Mai întâi, evenimentul original este serializat JSON cu `ensure_ascii=True`. Apoi, stringul JSON este encodat base64, pentru a putea fi împărțit și reasamblat fără probleme de caractere speciale. Rezultatul base64 este tăiat în N segmente, unde N depinde de dimensiunea payload-ului:

$$N = \left\lceil \frac{\text{lungime payload base64}}{8192} \right\rceil$$

Fiecare segment este trimis ca un eveniment nou de tip `annotation.chunk`. Evenimentul original nu mai este trimis direct; în schimb, serverul primește toate pachetele și reconstruiește evenimentul logic inițial. Payload-ul unui pachet conține identificatorul comun al transferului, sesiunea, tipul original, indexul pachetului, numărul total, encodarea și fragmentul de date.

```
{
  "type": "annotation.chunk",
  "payload": {
    "chunkId": "uuid-transfer",
    "sessionId": "abc12",
    "originalType": "annotation.update",
    "index": 0,
    "total": 4,
    "encoding": "base64",
    "data": "eyJldmVudElkIjoiLi4u"
  }
}
```

Pe server, `CollaborationWsController` detectează evenimentele de tip `annotation.chunk` și le transmite către `AnnotationChunkAssemblyService`. Serviciul parsează payload-ul în `AnnotationChunkPayloadDto` și validează câmpurile obligatorii: `chunkId`, `sessionId`, `originalType`, `index`, `total`, `encoding` și `data`. `index` trebuie să fie pozitiv sau zero, `total` trebuie să fie mai mare ca zero, iar `encoding` trebuie să fie `base64`. De asemenea, `sessionId` din payload trebuie să coincidă cu `sessionId` din envelope.

Starea temporară a transferului este păstrată într-un `ConcurrentHashMap`, indexat

după `chunkId` și `sessionId`. Pentru același transfer, serverul verifică faptul că toate pachetele au aceeași sesiune, același tip original, aceeași encodare și același număr total de pachete. Serverul verifică și faptul că secvența este trimisă de același actor. Dacă un pachet duplicat are același index și aceeași valoare, este acceptat ca duplicat inofensiv; dacă are același index, dar date diferite, transferul este respins.

Există și limite de protecție. Configurația modulului definește un TTL implicit de PT45S pentru transferurile incomplete și o limită implicită de 3000000 caractere pentru payload-ul total. Dacă un transfer nu se completează la timp, este curățat periodic. Dacă totalul de caractere depășește limita, serverul oprește reasamblarea. În plus, worker-ul STOMP de colaborare are o limită de siguranță de 2MB pentru un frame trimis direct; mesajele prea mari sunt evitate înainte de a provoca reconectări repetate.

Când toate pachetele au fost primite, serverul le concatenează în ordinea indexurilor, decodează base64, parsează JSON-ul rezultat ca `WsEventEnvelope` și verifică faptul că tipul decodat coincide cu `originalType`. Evenimentul reconstituit este apoi tratat ca un eveniment normal de adnotare: se verifică apartenența utilizatorului, se aplică modificarea în `CollaborationService`, se incrementează versiunea și se publică evenimentul final pe `/topic/sessions/{sessionId}`.

După aplicare, serverul trimite către client un răspuns personal pe coada de chunking. Dacă totul a reușit, tipul este `annotation.chunk.ack`; dacă pachetul este invalid sau aplicarea eșuează, tipul este `annotation.chunk.error`. Clientul ignoră evenimentele brute `annotation.chunk` primite din topic, dar tratează ack-ul și eroarea. La eroare, dacă transferul mai are încercări disponibile, clientul retrimite evenimentul original ca o nouă secvență de pachete; după două încercări eșuate, utilizatorul primește mesajul că sincronizarea a eșuat.

Acest mecanism permite colaborării să suporte atât adnotări mici, precum dreptunghiuri și poligoane, cât și măști dense, fără a schimba modelul logic de eveniment. Pentru restul sistemului, o mască actualizată rămâne tot un `annotation.update`; chunking-ul este doar stratul de transport care face posibilă livrarea sigură a payload-ului mare.

5.6. Serviciile backend și infrastructura Docker

Serviciile backend sunt pornite împreună prin `docker-compose.yml`. Compoziția definește un reverse proxy Traefik, un broker RabbitMQ, o bază Postgres pentru autentificare și cele trei servicii Spring Boot: `auth-service`, `chat-service` și `annotation-service`. Fiecare serviciu are propriul `Dockerfile`, iar configurarea sensibilă sau variabilă este transmisă prin fișierul `.env` și variabile de mediu.

Componentă	Port	Rol
reverse-proxy	80, 8081	Traefik, punct unic de intrare și dashboard local.
auth-service	8080	Gestionează autentificarea și rutele <code>/auth</code> .
chat-service	8084	Gestionează chatul, prezența și endpoint-ul <code>/ws</code> .
annotation-service	8085	Gestionează sesiunile, imaginea temporară și colaborarea pe adnotări.
rabbitmq	5672, 15672	Broker AMQP și consolă de management.
postgres-auth	5432	Baza persistentă pentru utilizatorii modului de autentificare.

Tabela 5.1: Componentele backend pornite prin Docker Compose

Traefik este punctul unic de intrare pentru backend. În `dynamic.yml`, ruta `/auth` este trimisă direct către `auth-service`, deoarece loginul și registerul trebuie să fie accesibile înainte ca utilizatorul să aibă token. Rutele `/chat`, `/notifications`, `/ws`, `/monitor` și `/annotation` trec prin middleware-ul `auth-forward`. Acest middleware apelează `http://auth-service:8080/auth/validate`; dacă token-ul este valid, cererea continuă către serviciul țintă, iar header-ul `X-User-Id` poate fi transmis mai departe.

```

/auth      -> auth-service:8080
/ws        -> chat-service:8084
/chat      -> chat-service:8084
/annotation -> annotation-service:8085

```

Pentru `/annotation`, Traefik aplică și middleware-ul `annotation-strip-prefix`. Acesta elimină prefixul `/annotation` înainte ca cererea să ajungă la serviciul Spring Boot. De aceea, clientul folosește ruta publică `/annotation/sessions`, dar `SessionController` este mapat intern la `/sessions`. Această soluție permite păstrarea unor rute publice clare, fără ca serviciul intern să fie obligat să cunoască prefixul de proxy.

RabbitMQ este folosit pentru evenimente între servicii, nu pentru mesajele live de colaborare din canvas. În implementarea curentă, cel mai clar exemplu este evenimentul de înregistrare a utilizatorului. `auth-service` publică evenimentul în exchange-ul `auth.events`, iar `chat-service` îl consumă prin coada `chat.user.register.queue`. `annotation-service` are de asemenea configurare RabbitMQ, ceea ce păstrează posibilitatea extinderii mecanismului de sincronizare între servicii, chiar dacă starea colaborativă live este gestionată prin WebSocket.

Postgres este folosit doar pentru zona de autentificare. În `docker-compose.yml`, serviciul `postgres-auth` pornește imaginea `postgres:15`, creează baza de auth, folosește un volum persistent și rulează scripturi de inițializare din directorul modulului de autentificare. `auth-service` depinde de healthcheck-ul Postgres și primește URL-ul JDBC prin variabile de mediu. Această dependență face ca serviciul de auth să pornească numai după ce baza poate accepta conexiuni.

Serviciile `chat-service` și `annotation-service` sunt expuse doar în rețeaua Docker prin `expose`, nu prin porturi publice directe. Clientul desktop le accesează prin Traefik, folosind `localhost` ca bază publică. Această alegere simplifică configurarea clientului: în loc ca aplicația să cunoască trei porturi diferite, ea poate folosi același host, iar proxy-ul decide serviciul țintă în funcție de prefix.

Din punct de vedere operațional, Docker Compose oferă o structură reproductibilă pentru backend. Se pornesc serviciile, brokerul și baza de date în aceeași rețea, cu aceleași nume DNS interne. Spring Boot primește configurația prin variabile de mediu, iar codul clientului poate rămâne orientat spre endpoint-uri stabile. Această structură nu transformă aplicația într-un sistem distribuit complex, dar separă clar responsabilitățile și permite testarea independentă a fiecărui serviciu.

5.7. Persistența și structura datelor

Persistența datelor este împărțită în funcție de natura fiecărui tip de informație. Nu toate datele au aceeași durată de viață. Conturile utilizatorilor trebuie să supraviețuiască restarturilor, conversațiile sunt utile în timpul rulării serviciului de chat, sesiunile colaborative sunt stări live, iar datasetul final este produs local prin export din clientul desktop. Această împărțire evită salvarea inutilă a unor date temporare și păstrează mai clar rolul fiecărei componente.

Pentru autentificare se folosește Postgres. Aici sunt salvate conturile, emailurile și parolele criptate. Această informație este persistentă și are nevoie de consistență între porniri, deoarece utilizatorul trebuie să poată reveni în aplicație cu același cont. Volumul Docker `postgres-auth-data` păstrează datele bazei între rulări, iar JPA gestionează entitățile modului de auth.

Pentru chat, serviciul folosește H2 în memorie pentru repository-ul de utilizatori sincronizați și o structură `ConcurrentHashMap` pentru istoricul conversațiilor. Mesajele private nu sunt salvate în Postgres; ele rămân în memoria serviciului și sunt limitate la ultimele 200 de mesaje per conversație. Această alegere este potrivită pentru prototipul colaborativ, deoarece chatul este un instrument de coordonare în timpul lucrului, nu arhiva principală a proiectului.

Pentru colaborare, `annotation-service` este explicit configurat ca modul in-memory. În `application.properties`, auto-configurările JDBC și JPA sunt excluse, iar starea este păstrată în obiecte Java: `sessionsById`, adnotările pe sesiune, utilizatorii prezenți, metadatele și conținutul temporar al imaginilor, plus evenimentele procesate pentru deduplicare. Sesiunile goale și inactive sunt curățate după un TTL implicit de cinci minute, imaginile temporare au TTL de patru ore, iar evenimentele procesate sunt păstrate pentru deduplicare timp de zece minute.

Această alegere are un compromis clar. Pe de o parte, colaborarea live este rapidă și ușor de urmărit în cod, deoarece serverul nu trebuie să scrie fiecare mișcare într-o bază de date. Pe de altă parte, o repornire a `annotation-service` pierde sesiunile active. În contextul proiectului, acest compromis este acceptabil deoarece artefactul final nu este sesiunea live, ci exportul local realizat de client: JSON intern, COCO, YOLO sau imagine cu măști. Serverul ajută utilizatorii să lucreze împreună, dar nu înlocuiește mecanismul de export al aplicației desktop.

Zonă de date	Mecanism folosit
Conturi utilizatori	Postgres, prin <code>auth-service</code> .
Utilizatori disponibili în chat	H2 în memorie, populat din evenimente RabbitMQ.
Istoric conversații	Memorie, maximum 200 mesaje per conversație.
Sesiuni colaborative	Memorie, cu versiuni, utilizatori și adnotări pe sesiune.
Imagini partajate temporar	Memorie, prin <code>TempImageStore</code> , cu token și TTL.
Dataset final	Fișiere locale exportate de <code>AnnotationEngine</code> .

Tabela 5.2: Persistența datelor pe componente

Separarea persistenței ajută și la explicarea limitelor sistemului. Dacă un utilizator se deloghează și revine, contul său rămâne în Postgres. Dacă serviciul de chat repornește, istoricul volatil se pierde, dar utilizatorii se pot resincroniza prin evenimentele de register și prin autentificare. Dacă o sesiune colaborativă este pierdută, adnotările trebuie recuperate din fișierele locale sau din exporturile realizate de client. Astfel, sistemul este proiectat ca un instrument de lucru și colaborare, nu ca o platformă completă de versionare permanentă a dataseturilor.

5.8. Rulare și integrarea componentelor

Pentru rularea sistemului complet sunt necesare două zone pornite separat: zona server din `ColaborativeServer` și clientul desktop din `AnnotationEngine`. Serviciile server se pornesc prin Docker Compose, ceea ce aduce în aceeași rețea Traefik, RabbitMQ, Postgres și cele trei servicii Spring Boot. Clientul desktop se rulează local și folosește endpoint-urile configurate în `QSettings` pentru autentificare, chat și colaborare.

Fluxul complet începe de obicei cu înregistrarea sau autentificarea utilizatorului. Clientul trimite cererea către ruta publică de login sau register, iar după autentificare token-ul este salvat local și devine baza pentru celelalte conexiuni. Când utilizatorul deschide fereastra de chat, worker-ul STOMP se conectează la endpoint-ul WebSocket de chat. Când utilizatorul folosește colaborarea pe adnotări, clientul REST accesează sesiunile din `/annotation/sessions`, iar worker-ul colaborativ se conectează la `/annotation/ws`.

Într-un scenariu colaborativ tipic, primul utilizator creează o sesiune, iar clientul trimite către backend identificatori pentru proiect, imagine și cameră. Dacă există o imagine activă în aplicație, ea poate fi încărcată temporar prin `/annotation/media/upload-temp`. Serverul publică apoi `image.available`, iar alți clienți pot descărca imaginea prin URL-ul temporar. După alăturare, fiecare client cere snapshot-ul sesiunii, aplică starea locală și începe să primească evenimente live.

Atunci când un utilizator creează, modifică sau șterge o adnotare, `MainWindow` transformă schimbarea locală într-un payload colaborativ. Acest payload ajunge în `ChatWindow`, care îl trimite prin `CollabStompWorker`. Pentru evenimente mici, trimiterea este directă. Pentru măști și payload-uri mari, se activează mecanismul de trimitere în N pachete. Serverul aplică evenimentul, îl versionază și îl publică înapoi către toți clienții din sesiune. Clienții care primesc evenimentul îl aplică pe `AnnotationManager`, dar folosesc flag-ul `_applying_remote_collab` pentru a nu retrimite aceeași modificare.

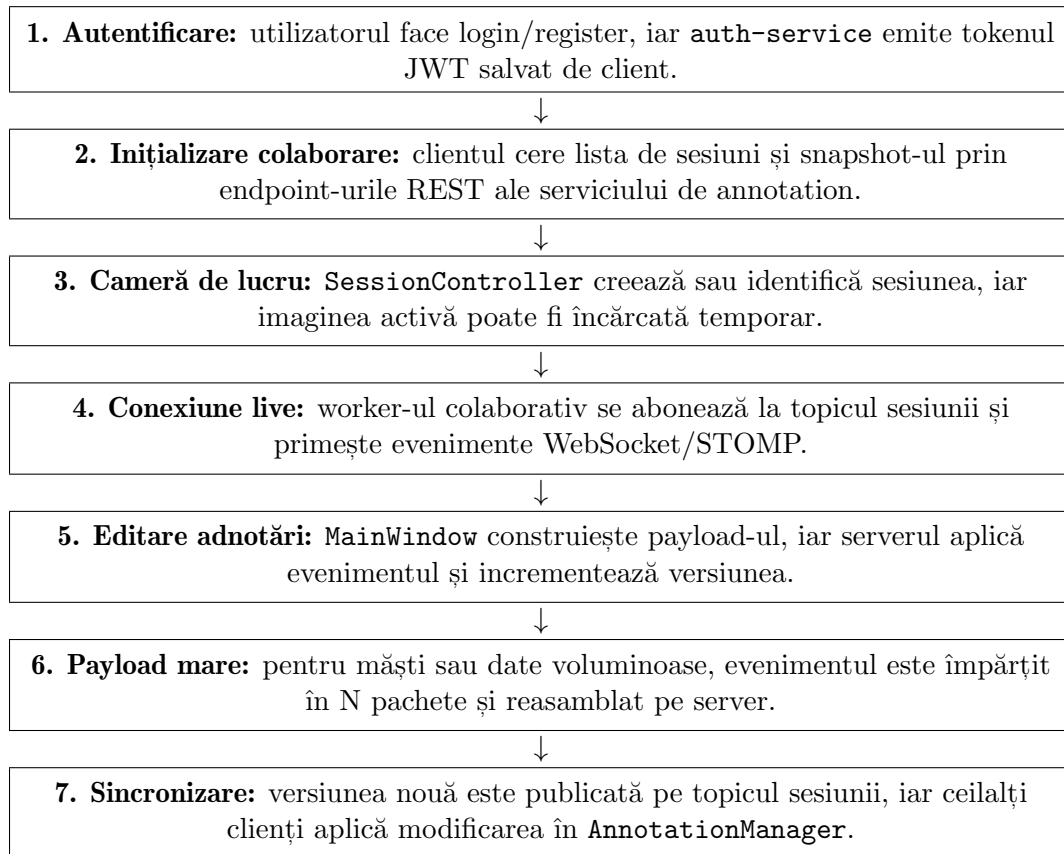


Figura 5.13: Fluxul operațional al colaborării dintre utilizator, clientul desktop și backend

Modelele AI și chatbotul local se configurează separat de backend-ul Docker. Auto-Seg și AutoMask pot fi executabile sau scripturi externe, iar căile lor sunt păstrate în **QSettings**. Ollama trebuie să ruleze local dacă utilizatorul alege backend-ul Ollama pentru chatbot, iar llama.cpp folosește un fișier local **GGUF**. Această separare este intenționată: backend-ul gestionează identitate și colaborare, în timp ce modelele AI sunt resurse locale controlate de client.

Prin această organizare, aplicația poate fi folosită gradual. Pentru adnotare individuală și export, utilizatorul are nevoie doar de clientul desktop și de fișierele locale. Pentru autentificare, chat și colaborare, se pornește backend-ul Docker. Pentru segmentare automată și chatbot, se configurează executabilele sau serviciile locale corespunzătoare. Sistemul nu obligă toate componentele să fie active pentru fiecare scenariu, dar le poate lega într-un flux complet atunci când utilizatorul lucrează colaborativ pe aceeași imagine.

Capitolul 6. Testare și validare

Împreună cu cele două capitole **precedente** trebuie să reprezinte aproximativ 70% din total.

Capitolul 7. Manual de instalare și utilizare

În secțiunea de Instalare trebuie să detaliați resursele software și hardware necesare pentru instalarea și rularea aplicației, precum și o descriere pas cu pas a procesului de instalare.

Instalarea aplicației trebuie să fie posibilă pe baza a ceea ce se scrie aici.

În acest capitol trebuie să descrieți cum se utilizează aplicația din punct de vedere al utilizatorului, fără a menționa aspecte tehnice interne.

Folosiți capturi ale ecranului și explicații pas cu pas ale interacțiunii.

Folosind acest manual, o persoană ar trebui să poată utiliza produsul vostru.

[Între 1 și 5 pagini.](#)

Capitolul 8. Concluzii

Între 1 și 2 pagini.

Capitolul ar trebui sa conțină (nu se rezumă neapărat la):

- un rezumat al contribuțiilor voastre
- analiză critică a rezultatelor obținute
- descriere a posibilelor dezvoltări și îmbunătățiri ulterioare

Bibliografie

- [1] A. Breitenfeld and C. Muller-Birn, “A state-of-the-art review of semantic annotation tools,” <http://dx.doi.org/10.17169/refubium-25139>, 2017.
- [2] M. Everingham, L. Gool, C. K. Williams, J. Winn, and A. Zisserman, “The pascal visual object classes (voc) challenge,” *Int. J. Comput. Vision*, vol. 88, no. 2, p. 303–338, Jun. 2010. [Online]. Available: <https://doi.org/10.1007/s11263-009-0275-4>
- [3] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, “Microsoft coco: Common objects in context,” 2015. [Online]. Available: <https://arxiv.org/abs/1405.0312>
- [4] A. Torralba and A. A. Efros, “Unbiased look at dataset bias,” in *CVPR 2011*, 2011, pp. 1521–1528.
- [5] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” 2015. [Online]. Available: <https://arxiv.org/abs/1505.04597>
- [6] B. Hariharan, P. Arbeláez, R. Girshick, and J. Malik, “Simultaneous detection and segmentation,” 2014. [Online]. Available: <https://arxiv.org/abs/1407.1808>
- [7] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár, “Panoptic segmentation,” 2019. [Online]. Available: <https://arxiv.org/abs/1801.00868>
- [8] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, “Masked-attention mask transformer for universal image segmentation,” 2022. [Online]. Available: <https://arxiv.org/abs/2112.01527>
- [9] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [10] F. Liang, B. Wu, X. Dai, K. Li, Y. Zhao, H. Zhang, P. Zhang, P. Vajda, and D. Marculescu, “Open-vocabulary semantic segmentation with mask-adapted clip,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.04150>
- [11] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, “Yoloe: Real-time seeing anything,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.07465>
- [12] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” 2023. [Online]. Available: <https://arxiv.org/abs/2304.08485>
- [13] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter,” 2020. [Online]. Available: <https://arxiv.org/abs/1910.01108>

-
- [14] B. Settles, “Active learning literature survey,” University of Wisconsin–Madison, Computer Sciences Technical Report 1648, 2009.
 - [15] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” *International Journal of Computer Vision*, vol. 128, no. 2, p. 336–359, Oct. 2019. [Online]. Available: <http://dx.doi.org/10.1007/s11263-019-01228-7>
 - [16] L. Castrejon, K. Kundu, R. Urtasun, and S. Fidler, “Annotating object instances with a polygon-rnn,” 2017. [Online]. Available: <https://arxiv.org/abs/1704.05548>
 - [17] K.-K. Maninis, S. Caelles, J. Pont-Tuset, and L. V. Gool, “Deep extreme cut: From extreme points to object segmentation,” 2018. [Online]. Available: <https://arxiv.org/abs/1711.09081>
 - [18] B. Chen, H. Ling, X. Zeng, G. Jun, Z. Xu, and S. Fidler, “Scribblebox: Interactive annotation framework for video object segmentation,” 2020. [Online]. Available: <https://arxiv.org/abs/2008.09721>
 - [19] A. Torralba, B. C. Russell, and J. Yuen, “Labelme: Online image annotation and applications,” *Proceedings of the IEEE*, vol. 98, no. 8, pp. 1467–1484, 2010.
 - [20] C. contributors, “Cvat: Computer vision annotation tool,” GitHub repository, 2025. [Online]. Available: <https://github.com/cvat-ai/cvat>
 - [21] C. Sun, D. Sun, A. Ng, W. Cai, and B. Cho, “Real differences between ot and crdt under a general transformation framework for consistency maintenance in co-editors,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, no. GROUP, pp. 1–26, 2020. [Online]. Available: <https://doi.org/10.1145/3375186>
 - [22] “Stomp protocol specification, version 1.2,” STOMP Project. [Online]. Available: <https://stomp.github.io/stomp-specification-1.2.html>
 - [23] J. James, “Counting on consensus: Selecting the right inter-annotator agreement metric for nlp annotation and evaluation,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.06865>
 - [24] L. Moreau and P. Missier, “Prov-dm: The prov data model,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
 - [25] “Postgresql documentation,” PostgreSQL Global Development Group. [Online]. Available: <https://www.postgresql.org/docs/>
 - [26] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge.” New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: <https://doi.org/10.1145/3037697.3037698>
 - [27] X. Wang, H. Zhang, M. Yang, X. Wu, and P. Cheng, “Privacy-preserving collaborative learning: A scheme providing heterogeneous protection,” *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 1840–1853, 2024.

- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [29] G. Gerganov and contributors, “llama.cpp,” GitHub repository, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [30] O. contributors, “Ollama,” Local model runtime and client, 2026. [Online]. Available: <https://ollama.com/>

Anexa A. Sursele și proveniența figurilor

Tabelele de mai jos indică sursele și proveniența imaginilor folosite în lucrare. Numele figurii este un link intern către figura corespunzătoare din capitolul în care este folosită.

Tabela A.1: Sursele figurilor folosite în capitolul de analiză

Figură	Link sursă
Figura 4.1 – Arhitectura YOLOE	Sursă YOLOE
Figura 4.2 – Arhitectura Mask2Former	Sursă Mask2Former
Figura 4.3 – Transformer decoder-only	Sursă Transformer
Figura 4.4 – Structura JWT	Sursă JWT

Tabela A.2: Proveniența figurilor folosite în capitolul de proiectare și implementare

Figură	Proveniență
Figura 5.1 – Arhitectura generală a aplicației	-
Figura 5.2 – Interfața principală a aplicației PyVisionAnnotator	-
Figura 5.3 – Organizarea internă a ferestrei principale	-
Figura 5.4 – Panoul de setări pentru AutoSeg și AutoMask	-
Figura 5.5 – Panoul de proprietăți și lista de adnotări	-
Figura 5.6 – Fluxul de evenimente în canvas	-
Figura 5.7 – Clasele principale pentru adnotări	-
Figura 5.8 – Fluxul AutoSeg YOLO în client	-
Figura 5.9 – Fluxul AutoMask pe baza unui punct selectat	-
Figura 5.10 – Fereastra de chat și colaborare	-
Figura 5.11 – Fereastra chatbot local cu Ollama sau llama.cpp	-
Figura 5.12 – Fluxul de încărcare imagine și protecția modificărilor nesalvate	-
Figura 5.13 – Fluxul operațional al colaborării dintre utilizator, clientul desktop și backend	-